SAVITRIBAI PHULE PUNE UNIVERSITY

A PROJECT REPORT ON

AI FOR LOST CHILD IDENTIFICATION

SUBMITTED TOWARDS THE PARTIAL
FULFILLMENT OF THE REQUIREMENTS OF

BACHELOR OF ENGINEERING (Computer Engineering)

BY

Name: Janhavi Mahajan	**Seat No.: B401020173**
Name: Gayatri Kadam	**Seat No.: B401020155**
Name: Tanisha Kahane	**Seat No.: B401020153**
Name: Isha Soni	**Seat No.: B401020146**

Under the Guidance of
Mrs. Shafali Gupta

Siinhgad Instiitutes

DEPARTMENT OF COMPUTER ENGINEERING

RMD SINHGAD SCHOOL OF ENGINEERING

WARJE, PUNE – 411058

2025 – 26

Siinhgad Instiitutes

RMD SINHGAD SCHOOL OF ENGINEERING

DEPARTMENT OF COMPUTER ENGINEERING

CERTIFICATE

This is to certify that the project report entitled

AI FOR LOST CHILD IDENTIFICATION

Submitted by

Name: Janhavi Mahajan

Name: Gayatri Kadam

Name: Tanisha Kahane

Name: Isha Soni

is a bonafide work carried out by them under the supervision of Mrs. Shafali Gupta and it is submitted towards the partial fulfilment of the requirement of Bachelor of Engineering (Computer Engineering).

Mrs. Shafali Gupta
Internal Guide
Dept. of Computer Engg.

Head Of Department
Dept. of Computer Engg.

Dr. V. V. Dixit
Principal
RMD Sinhgad School of Engineering

Signature of Internal Examiner

Signature of External Examiner

PROJECT APPROVAL SHEET

AI FOR LOST CHILD IDENTIFICATION

Is successfully completed by

Name: Janhavi Mahajan	**Seat No.: B401020173**
Name: Gayatri Kadam	**Seat No.: B401020155**
Name: Tanisha Kahane	**Seat No.: B401020153**
Name: Isha Soni	**Seat No.: B401020146**

at

DEPARTMENT OF COMPUTER ENGINEERING

RMD SINHGAD SCHOOL OF ENGINEERING

SAVITRIBAI PHULE PUNE UNIVERSITY,PUNE

ACADEMIC YEAR 2025-2026

Mrs. Shafali Gupta

Internal Guide

Dept. of Computer Engg.

Head Of Department

Dept. of Computer Engg.

ABSTRACT

Child safety has become a significant global concern, especially in crowded public places such as parks, railway stations, shopping malls, and airports, where incidents of missing or abducted children are increasing. Traditional CCTV-based surveillance systems are often ineffective due to human fatigue, limited monitoring capability, and delayed response. To overcome these limitations, this work proposes an AI-driven child detection and recognition framework using MTCNN, MediaPipe, and FaceNet for automated monitoring and identification.

The proposed system utilizes MTCNN (Multi-task Cascaded Convolutional Neural Network) for accurate detection of human faces in real-time video streams. MTCNN not only detects faces but also identifies key facial landmarks, improving detection precision in unconstrained environments. The detected faces are further refined and cropped using MediaPipe for enhanced face localization. These facial regions are then processed using FaceNet, which converts them into high-dimensional embeddings for comparison with a pre-existing database of known children. If the similarity score exceeds a predefined confidence threshold, the system generates an alert displaying the child's identity and matching percentage.

The dataset used in this work consists of both real and AI-generated synthetic child faces, enabling better diversity and improved generalization of the model. The system is evaluated on simulated CCTV-style video streams and achieves an average recognition accuracy of 91.8%, while maintaining near real-time performance on standard hardware systems.

The framework is capable of analyzing video input, detecting multiple individuals, and identifying whether a known child from the database is present in the frame. Upon successful identification, it displays outputs such as "Detected Child: Rahul – 92% match." Synthetic data and simulation videos are generated using AI tools such as Runway ML and Pika Labs to enhance testing scenarios.

By automating the detection and recognition process, the proposed system transforms traditional CCTV systems into intelligent surveillance solutions. It has strong potential applications in smart cities, transportation hubs, and public safety systems, assisting law enforcement agencies in real-time monitoring and child protection.

Overall, this approach demonstrates how deep learning and computer vision techniques can be effectively utilized to improve public safety, enable rapid identification of missing children, and provide a scalable and efficient solution to a critical societal issue.

Keywords: Child detection, face recognition, MTCNN, MediaPipe, FaceNet, deep learning, computer vision.

ACKNOWLEDGEMENT

It gives us immense pleasure to present the preliminary project report on **"AI For Lost Child Identification System."** We would like to take this opportunity to express our heartfelt gratitude to our internal guide **Mrs. Shafali Gupta** for her constant support, encouragement, and valuable guidance throughout the development of this project. We are truly grateful for her kind assistance and constructive suggestions, which were instrumental in the successful completion of our work.

We also extend our sincere thanks to Head of the Department of Computer Engineering, **RMD Sinhgad School of Engineering (RMDSSOE)**, for their indispensable support, encouragement, and valuable suggestions. We are thankful to all the faculty members of the Computer Engineering Department for their continuous assistance and insightful guidance during the course of this project. In the end, our special thanks to the institution for providing the necessary resources, laboratory facilities, software platforms, and continuous Internet connectivity required for the successful development of our project.

Janhavi Mahajan
Gayatri Kadam
Isha Soni
Tanisha Kahane

B.E. (Computer Engineering)

INDEX

1. SYNOPSIS **1**

1.1 PROJECT TITLE....1

1.2 PROJECT OPTION....1

1.3 INTERNAL GUIDE....1

1.4 SPONSORSHIP & EXTERNAL GUIDE....1

1.5 TECHNICAL KEYWORDS (AS PER ACM KEYWORDS)....1

1.6 PROBLEM STATEMENT....2

1.7 ABSTRACT....2

1.8 GOALS & OBJECTIVES....3

1.9 RELEVANT MATHEMATICS ASSOCIATED WITH THE PROJECT....3

1.10 NAMES OF CONFERENCES/JOURNALS WHERE PAPERS CAN BE PUBLISHED....6

1.11 REVIEW OF CONFERENCE/JOURNAL PAPERS SUPPORTING PROJECT IDEA6

1.12 PLAN OF PROJECT EXECUTION....7

2. TECHNICAL KEYWORDS **8**

2.1 AREA OF PROJECT....9

2.2 TECHNICAL KEYWORDS....9

3. INTRODUCTION **11**

3.1 PROJECT IDEA....12

3.2 MOTIVATION OF THE PROJECT....12

3.3 LITERATURE SURVEY....13

4. PROBLEM DEFINITION AND SCOPE **15**

4.1 PROBLEM STATEMENT....16

4.1.1 Goals & objectives....16

4.1.2 Statement of Scope....16

4.2 MAJOR CONSTRAINTS....18

4.3 METHODOLOGIES OF PROBLEM SOLVING AND EFFICIENCY ISSUES....19

4.3.1 SCENARIOS IN WHICH MULTI-CORE, EMBEDDED AND DISTRIBUTED COMPUTING USED....20

4.4 OUTCOME 20
4.5 APPLICATIONS 21
4.6 HARDWARE RESOURCES REQUIRED 22
4.7 SOFTWARE RESOURCES REQUIRED 23

5 PROJECT PLAN 24
5.1 PROJECT ESTIMATES 25
5.1.1 Reconciled Estimates 25
5.1.2 Project Resources 26
5.2 RISK MANAGEMENT W.R.T. NP – HARD ANALYSIS 26
5.2.1 Risk Identification 27
5.2.2 Risk Analysis 27
5.2.3 Overview of Risk Mitigation, Monitoring, Management 29
5.3 PROJECT SCHEDULE 30
5.3.1 Project Task set 30
5.3.2 Task Network 31
5.3.3 Timeline Chart 31
5.4 TEAM ORGANISATION 32
5.4.1 Team Structure 32
5.4.2 Management Reporting and Communication 32

6 SOFTWARE REQUIREMENT SPECIFICATION 33
6.1 INTRODUCTION 34
6.1.1 Purpose and Scope of Document 34
6.1.2 Overview of responsibilities 34
6.2 USAGE SCENARIO 35
6.2.1 User Profiles 35
6.2.2 Use Cases 37
6.2.3 Use Case View 38
6.3 DATA MODEL AND DESCRIPTION 38
6.3.1 Data Description 38
6.3.2 Data Objects and Relationships 38
6.4 FUNCTIONAL MODEL AND DESCRIPTION 39

6.4.1 Data Flow Diagram 39
6.4.2 Activity Diagram 41
6.4.3 Non-Functional Requirements 41
6.4.4 State Diagram 42
6.4.5 Design Constraints 42
6.4.6 Software Interface Description 43

7 DETAILED DESIGN DOCUMENT USING APPENDIX A AND B 45
7.1 INTRODUCTION 46
7.2 ARCHITECTURAL DESIGN 46
7.3 DATA DESIGN (USING APPENDICES A AND B) 47
7.3.1 Internal Software Data Structure 47
7.3.2 Global Data Structure 48
7.3.3 Temporary Data Structure 48
7.3.4 Database Description 48
7.4 COMPONENT DESIGN 49
7.4.1 Class Diagram 49
7.4.2 Sequence Diagram 50

8 Project Implementation 51
8.1 INTRODUCTION 52
8.2 Tools and Technologies Used 52
8.3 Methodologies/Algorithm Details) 52
8.3.1 Algorithm 1/Pseudo Code 53
8.3.2 Algorithm 2/Pseudo Code 57
8.4 Verification and Validation for Acceptance 61

9 Software Testing 62
9.1 Type of Testing Used 63
9.2 Test Cases and Test Results 64

10 Results 65
10.1 Screen shorts 66

10.2 Outputs 67

11 Deployment and Maintenance 69

11.1 Installation and un-installation..... 70

11.2 User help..... 71

12 Conclusion and future scope 72

12.1 Conclusion……..73

12.2 Future Scope……...74

Annexure A References 75

Annexure B Project Planner 79

Annexure C Reviewers Comments of Paper Submitted 82

Annexure D Plagiarism Report 84

Annexure E Information of Project Group Members 86

LIST OF FIGURES

Figure 1: Task Network Flow.....31
Figure 2: Timeline as per Gantt Chart.....31
Figure 3: Use Case Diagram.....38
Figure 4: Data Flow Diagram (DFD Level 0).....39
Figure 5: Data Flow Diagram (DFD Level 1).....39
Figure 6: Data Flow Diagram (DFD Level 2).....40
Figure 7: Data Flow Diagram (DFD Level 3).....40
Figure 8: Activity Diagram.....41
Figure 9: State Diagram.....42
Figure 10: System Architecture Design.....46
Figure 11: Class Diagram.....49
Figure 12: Sequence Diagram.....50
Figure 13: Gantt Chart for Project.....81

LIST OF TABLES

Table 1: Hardware Requirements 22
Table 2: Cost Estimates 25
Table 3: Time Estimates 25
Table 4: Risk Table 28
Table 5: Risk Probability Definitions 28
Table 6: Risk Impact Definitions 28
Table 7: Risk Delay in Component Procurement 29
Table 8: Risk in Inadequate Sensor Accuracy 29
Table 9: Use Case 37
Table 10: Test Cases 64

CHAPTER 1
SYNOPSIS

1.1 PROJECT TITLE

AI For Lost Child Identification

1.2 PROJECT OPTION

Internal Project

1.3 INTERNAL GUIDE

Mrs. Shafali Gupta

1.4 SPONSORSHIP & EXTERNAL GUIDE

Not Applicable

1.5 TECHNICAL KEYWORDS (AS PER ACM KEYWORDS)

A. Computer Systems Organisation

- A.1. Embedded and Cyber-Physical Systems
- A.2. Sensors and Actuators (CCTV Cameras, Video Inputs)
- A.3. Embedded Systems (AI-Based Surveillance Systems)

B. Software and Its Engineering

- B.1. Software Organisation and Properties
- B.2. Contextual Software Domains
- B.3. Operating Systems
- B.4. Communications Management
  (Communication between Detection Modules and Dashboard Interface for Real-Time Alert Processing)

C. Information Systems

- C.1. Information Systems Applications
- C.2. Multimedia Information Systems (Real-Time CCTV Footage Analysis)
- C.3. User Interfaces (Security Dashboard for Detection Alerts)

D. Computing Methodologies

- D.1. Machine Learning

D.2. Machine Learning Approaches (MTCNN , FaceNet, DeepFace for Object and Face Recognition)

D.3. Cross-Validation (Accuracy Measurement, Threshold Tuning for Face Matching)

1.6 PROBLEM STATEMENT

To develop an AI-based system that can detect and recognize known or missing children from live CCTV footage using computer vision and deep learning, providing real-time alerts to improve public safety and surveillance efficiency**.**

1.7 ABSTRACT

Child safety has become a significant global concern, especially in crowded public places such as parks, railway stations, shopping malls, and airports, where incidents of missing or abducted children are increasing. Traditional CCTV-based surveillance systems are often ineffective due to human fatigue, limited monitoring capability, and delayed response. To overcome these limitations, this work proposes an AI-driven child detection and recognition framework using MTCNN, MediaPipe, and FaceNet for automated monitoring and identification.

The proposed system utilizes MTCNN (Multi-task Cascaded Convolutional Neural Network) for accurate detection of human faces in real-time video streams. MTCNN not only detects faces but also identifies key facial landmarks, improving detection precision in unconstrained environments. The detected faces are further refined and cropped using MediaPipe for enhanced face localization. These facial regions are then processed using FaceNet, which converts them into high-dimensional embeddings for comparison with a pre-existing database of known children. If the similarity score exceeds a predefined confidence threshold, the system generates an alert displaying the child's identity and matching percentage.

The dataset used in this work consists of both real and AI-generated synthetic child faces, enabling better diversity and improved generalization of the model. The system is evaluated on simulated CCTV-style video streams and achieves an average recognition accuracy of 91.8%, while maintaining near real-time performance on standard hardware systems.

The framework is capable of analyzing video input, detecting multiple individuals, and identifying whether a known child from the database is present in the frame. Upon successful identification, it displays outputs such as "Detected Child: Rahul – 92% match." Synthetic data and simulation videos are generated using AI tools such as Runway ML and Pika Labs to enhance testing scenarios.

By automating the detection and recognition process, the proposed system transforms traditional CCTV systems into intelligent surveillance solutions. It has strong potential applications in smart cities, transportation hubs, and public safety systems, assisting law enforcement agencies in real-time monitoring and child protection.

Overall, this approach demonstrates how deep learning and computer vision techniques can be effectively utilized to improve public safety, enable rapid identification of missing children, and provide a scalable and efficient solution to a critical societal issue.

Keywords: Face recognition, MTCNN, MediaPipe, FaceNet, deep learning, computer vision.

1.8 GOALS & OBJECTIVES

- To detect and identify children in real-time from live CCTV or video footage using advanced AI models.
- To enhance existing surveillance systems by integrating automatic face detection and recognition features.
- To maintain a database of known children for accurate identity verification and quick alert generation.
- To provide a user-friendly dashboard displaying detected child names, confidence levels, and timestamps.
- To support authorities in locating missing children quickly and efficiently through smart AI-driven monitoring.
- To contribute to Smart City and public safety projects through reliable, scalable, and automated AI solutions.

1.9 RELEVANT MATHEMATICS ASSOCIATED WITH THE PROJECT

System Description:

- **Input:**
  Live video streams from CCTV cameras or video files containing multiple individuals (children and adults).
- **Output:**
  Real-time display of detected children, their identified names (if matched with the database), and confidence scores shown on the monitoring dashboard.

1. Centroid

$$\text{Centroid} = \frac{1}{N}\sum_{i=1}^{N} E_i$$

2. Bounding Box Regression

MTCNN predicts the location of a face by refining the bounding box.

$$B' = B + \Delta B$$

Where:

- $B = (x, y, w, h)$= Original bounding box
- $\Delta B = (\Delta x, \Delta y, \Delta w, \Delta h)$= Predicted correction
- B'= Refined bounding box

1. Data Structures and Classes

- Detection Data Structures:
  Data Type: Arrays or NumPy tensors are used to store pixel data from video frames for object detection and face recognition.
  Bounding Box Data: Lists of dictionaries or JSON objects store details such as detected object ID, class label ("child"), coordinates (x, y, width, height), and confidence score.
- Face Recognition Data Structures:
  Face Embedding Vectors: Stored in arrays or vectors of floating-point numbers (e.g., 128-dimensional vectors from FaceNet).
  Child Database: Dictionary or JSON-based structure mapping each child's ID to their name, image embeddings, and metadata.
- Communication Class:
  Manages data flow between detection modules (MTCNN + MediaPipe + FaceNet) and the dashboard interface using APIs.
- Detection Algorithm Classes:
  Encapsulates MTCNN detection, MediaPipe face cropping, and FaceNet/DeepFace recognition logic.
  Handles preprocessing, feature extraction, and real-time classification.
- Result Management Class:
  Responsible for logging recognized faces, storing match results, and generating alerts with timestamps.

2. Divide and Conquer Strategies

- Parallel Frame Processing:
  Split the incoming video stream into smaller frame batches, processed in parallel to improve detection speed.
- Face Detection:
  MTCNN is used for real-time face detection and landmark localization, while MediaPipe and FaceNet run simultaneously for face detection and recognition.
- Parallel Database Matching:
  Use multi-threading to compare multiple detected face embeddings with stored child embeddings to reduce recognition delay.
- Asynchronous Alert System:
  When a child is identified, alerts are sent asynchronously to the dashboard, ensuring detection continues smoothly without interruption.

3. Constraints

- Latency Constraints:
  The system must perform real-time detection and recognition with minimal delay (preferably under 200ms per frame).

- Resource Constraints:
  Optimization is needed for limited GPU/CPU availability — especially when processing high-resolution video streams.
- Accuracy Constraints:
  Ensuring high precision and recall in face matching even in low-light or crowded conditions.
- Scalability Constraints:
  The system should handle multiple camera inputs simultaneously without performance degradation.

4. Functions

- Face Detection and Cropping Function: Detects faces within detected bounding boxes using Media Pipe and crops them for recognition.
- Face Embedding and Comparison Function: Converts detected faces into embedding vectors and compares them with the stored embeddings using Euclidean distance or cosine similarity.
- Alert Generation Function: Generates and displays alerts such as "Detected Child: Rahul – 92% match" along with timestamps and location.
- Data Logging and Visualization Function: Records all detections, confidence levels, and frame details for visualization on a dashboard or web interface.

5. Success Conditions

- Accurate detection of children and reliable recognition of known faces with high confidence.
- Real-time alerts displayed on the monitoring dashboard with minimal latency.
- Robust performance under varying lighting, crowd density, and camera angles.
- Scalable and adaptable system capable of integrating with multiple CCTV sources.

6. Failure Conditions

- Incorrect detections (false positives/negatives), such as misidentifying adults as children.
- Poor recognition accuracy due to low-resolution or blurred faces.
- Delays in real-time alert generation caused by heavy processing load.
- System or hardware failure resulting in missed detections or incomplete frame analysis.
- Network or API issues interrupting communication between detection modules and the dashboard.

1.10 NAMES OF CONFERENCES / JOURNALS WHERE PAPERS CAN BE PUBLISHED

- IEEE International Conference on Wearable and Implantable Body Sensor Networks (BSN)
- IEEE Transactions on Biomedical Engineering
- Conferences / Workshops in IITs
- Central Universities or SPPU Conferences
- IEEE / ACM Conference

1.11 REVIEW OF CONFERENCE/JOURNAL PAPERS SUPPORTING PROJECT IDEA

Recent advancements in artificial intelligence and deep learning have significantly improved the performance of face detection and face recognition systems for public safety applications. Chai et al. [1] demonstrated that the MTCNN model effectively detects facial regions and landmarks under different lighting conditions, making it suitable for accurate face alignment. Dang and Tran [2] proposed a secure face recognition framework by integrating MTCNN with Face Net and head pose estimation, achieving improved recognition accuracy under varying face orientations. Wang and Liu [3] introduced a lightweight MTCNN architecture for real-time edge deployment, reducing computational complexity while maintaining detection performance. Pratama and Ningrum [4] combined MTCNN, ResNetV1, and SVM for face recognition but required retraining whenever new identities were added. Comparative studies on Face Net, Deep Face, and Open Face [5] showed that Face Net provides superior facial embeddings and higher recognition accuracy. Mensah et al. [6] evaluated the robustness of Face Net under multiple constraints and concluded that image quality greatly influences recognition performance. Several studies have also focused on missing child identification using machine learning and deep learning techniques [7], [8], highlighting the importance of automated identification systems for reducing search time. Patil et al. [9] developed a child monitoring system integrating face, emotion, and activity recognition to enhance child safety. Similarly, Venkatasalam et al. [10] proposed an AI-assisted missing person finder using Face Net for facial recognition. Based on the literature, the proposed system integrates MTCNN for face detection and alignment, Face Net for feature extraction, and cosine similarity matching to provide an efficient and scalable solution for real-time lost child identification.

1.12 PLAN OF PROJECT EXECUTION

Week	Start Date	Activity
1	05 January 2026	Project Initiation
2	14 January 2026	Research and Project Title Finalization
3	14 January 2026	First Project Review
4	22 January 2026	Base Paper and System Architecture
5	03 Februrary 2026	Project Review and Documentation
6	12 Februrary 2026	Analyzing hardware and software requirements
7	23 Februrary 2026	Existing System Analysis and Design
8	12 March 2026	Survey Report Documentation
9	23 March 2026	Project Stage-1 Report Documentation
10	06 April 2026	Project Report and UML Diagrams
11	24 April 2026	Documentation Approval and Publication
12	19 May 2026	Final Review and Analysis

CHAPTER 2
TECHNICAL KEYWORDS

2.1 AREA OF PROJECT

The project falls within the domain of Artificial Intelligence and Computer Vision, focusing on real-time surveillance and facial recognition for child safety applications. With the increasing number of missing child incidents in crowded public areas such as malls, railway stations, and parks, there is a growing need for intelligent systems that can automatically identify and alert authorities in real-time.

This project aims to transform conventional CCTV surveillance into an AI-powered detection system using advanced deep learning algorithms. It utilizes MTCNN for detection, Media Pipe for face detection and cropping, and FaceNet/DeepFace for face recognition. By processing live video feeds, the system can accurately identify known or missing children and instantly generate alerts.

Through the combination of computer vision, deep learning, and real-time data analytics, this project contributes to the Smart City and public safety domains. It demonstrates how AI can be leveraged not only for technological advancement but also for social impact, helping prevent child abductions, improving response times, and ensuring safer public environments.

2.2 TECHNICAL KEYWORDS

A. Computer Systems Organisation

A.1 Embedded and Cyber-Physical Systems

A.2 Sensors and Actuators (CCTV Cameras, IP Cameras, Video Streams)

A.3 Embedded Systems (AI-Integrated Surveillance Systems)

B. Software and its Engineering

B.1 Software Organisation and Properties

B.2 Contextual Software Domains

B.3 Operating Systems

B.4 Communications Management

(Communication between AI Detection Module and Monitoring Dashboard for Real-Time Alerts)

C. Information Systems

C.1 Information Systems Applications

C.2 Multimedia Information Systems (Real-Time Video Frame Analysis)

C.3 User Interfaces (Dashboard for Surveillance and Child Recognition Alerts)

D. Computing Methodologies

D.1 Machine Learning

D.2 Machine Learning Approaches (MTCNN, FaceNet, DeepFace Models for Detection and Recognition)

D.3 Cross-Validation (Confidence Scoring, Threshold Optimization, and Model Accuracy Evaluation)

CHAPTER 3
INTRODUCTION

3.1 PROJECT IDEA

The proposed project aims to develop an AI-based Lost Child Identification System that enhances public safety through the integration of advanced computer vision and deep learning technologies. In large public spaces such as malls, railway stations, amusement parks, and events, it is common for children to become separated from their guardians. Although most of these areas are equipped with CCTV cameras, existing systems only record footage and do not provide real-time identification or alert mechanisms. This project seeks to address that gap by transforming traditional surveillance systems into intelligent, proactive monitoring solutions.

The system leverages MTCNN for accurate face detection and landmark extraction within video streams, followed by MediaPipe for precise face localization and cropping, and FaceNet/DeepFace for recognizing and matching faces against a pre-stored child database. Once a match is found, the system displays the child's name and confidence level (e.g., "Detected Child: Rahul – 92% match"), providing instant alerts to security authorities.

To simulate real-world scenarios, both real and AI-generated datasets will be used, along with test videos created using tools like Runway ML and Pika Labs. The project not only demonstrates the power of artificial intelligence in solving real-world problems but also contributes to the Smart City vision by ensuring safer, more responsive public surveillance systems. Ultimately, it aims to assist in locating missing children quickly and efficiently, providing peace of mind to families and aiding law enforcement in real-time operations.

3.2 MOTIVATION OF THE PROJECT

Every year, thousands of children go missing in crowded public places due to human negligence or unforeseen circumstances. Despite the widespread presence of CCTV cameras, most surveillance systems today are passive, requiring human operators to manually monitor multiple screens—an inefficient and error-prone process. This delay in response can make it extremely difficult to locate a missing child quickly, increasing the risk of abduction or harm.

The motivation behind this project lies in using Artificial Intelligence to address this pressing societal problem. By integrating real-time detection and recognition algorithms, the proposed system automates the process of identifying and locating children in video feeds, drastically reducing response time and improving accuracy.

In addition to technological innovation, this project carries a strong humanitarian purpose. It aims to assist parents, security personnel, and law enforcement authorities by providing a reliable system that enhances public safety. The ability to automatically identify children through AI not only brings a sense of security but also helps prevent tragic incidents.

Through this initiative, the project team aspires to contribute toward building AI-driven Smart Cities that can actively protect vulnerable individuals, especially children. This motivation stems from the belief that technology should serve society—transforming ordinary surveillance systems into intelligent tools that save lives and create safer communities.

3.3 LITERATURE SURVEY

The study in [1] describes a three-step deep learning model for improving face detection under real-world conditions such as low light, different face angles, facial expressions, and occlusions like masks or glasses. It emphasizes that accurate face detection is essential because incorrect detection leads to poor face recognition. The first network identifies possible face regions, the second removes false detections and refines the face box, and the third confirms the face while detecting key facial landmarks such as the eyes, nose, and mouth. The model was trained on a large public dataset containing faces with different poses and lighting conditions. Compared to a single-network approach, the three-stage model achieved higher accuracy and faster detection. The study focuses on improving face detection, which strengthens the overall reliability and performance of face recognition systems.

The paper in [2] presents a face recognition system that combines head pose estimation, MTCNN, and FaceNet to improve accuracy and security. It addresses the limitations of traditional identification methods such as ID cards and fingerprint scanners, which are less secure, costly, and difficult to maintain. The system first verifies that the user is a real person by requiring five head poses (straight, left, right, up, and down), preventing spoofing attacks using photos. MTCNN is then used to detect and align the face, while FaceNet extracts facial features and converts them into feature vectors for matching with stored records. The system was tested on a Jetson Nano device and achieved an accuracy of 90.95% with real-time performance. The study concludes that combining head pose estimation, MTCNN, and FaceNet significantly improves the security, accuracy, and reliability of face recognition for real-world identification applications.

The study in [3] proposes an improved version of the traditional MTCNN face detection model. It explains that the original MTCNN has difficulty detecting small faces, side-angle faces, and faces in low-light conditions, while also requiring high computational power. To overcome these limitations, the authors replaced the internal convolution layers with Ghost bottleneck modules while retaining the three-stage architecture (P-Net, R-Net, and O-Net). This reduced computational cost without affecting detection performance. Experimental results on the WIDER FACE dataset showed improved accuracy and faster detection, especially on FPGA hardware. This work is relevant to our project because we also use MTCNN for face detection. However, our system uses the standard MTCNN model and focuses on face recognition using FaceNet embeddings and cosine similarity matching to identify missing children rather than optimizing the detection network for edge devices.

The study in [4] presents an improved MTCNN face detection model to overcome the limitations of the original model, such as poor detection of small faces, side-angle faces, and faces in low-light conditions. To reduce computational cost while maintaining the three-stage architecture (P-Net, R-Net, and O-Net), the authors introduced Ghost bottleneck modules. Experimental results on the WIDER FACE dataset showed that the improved model achieved higher detection accuracy and faster performance, making it suitable for resource-limited devices and FPGA

hardware. This work is related to our project because we also use MTCNN for face detection. However, our primary objective is to identify missing children using FaceNet embeddings and cosine similarity matching for face recognition and identity verification. Unlike systems that use SVM for classification, our approach uses threshold-based cosine similarity matching for reliable surveillance and missing child identification.

The study in [5] presents a comparative analysis of modern deep learning-based face recognition models, including FaceNet, DeepFace, and OpenFace. It explains the transition from traditional methods such as Eigenfaces to deep CNN-based models that generate high-dimensional facial embeddings for accurate face verification and identification. Among the compared models, FaceNet achieved the highest accuracy of 99.64%, followed by DeepFace (97.25%) and OpenFace (92.92%) on the LFW dataset. The paper also highlights the importance of image preprocessing in improving recognition performance. Although these models provide high accuracy, they have limitations such as high computational cost, the need for large annotated datasets, scalability challenges, and privacy and bias concerns.

The study in [6] evaluates the performance of the FaceNet algorithm under challenging conditions such as different facial expressions and 30–40% facial occlusions. It found that FaceNet's accuracy decreases when images with occlusions are used directly. To improve performance, the authors applied image reconstruction techniques such as MICE, MissForest, and RegEM, along with data augmentation methods including rotation, flipping, shearing, brightness adjustment, and stretching. Experimental results showed that these techniques significantly improved the robustness and accuracy of the FaceNet model, especially when training data was limited. The study concludes that reconstruction and data augmentation are effective approaches for enhancing FaceNet's performance in real-world face recognition applications.

The study in [7] focuses on improving the speed and accuracy of identifying missing children using face detection and recognition techniques such as Haar Cascade, HOG (Histogram of Oriented Gradients), and Convolutional Neural Networks (CNN). To improve image quality, especially for low-quality surveillance footage, the system uses CLAHE image enhancement before face detection. It also provides real-time monitoring, automated alerts, location tracking, and secure data storage through encryption. By integrating these technologies, the system enables faster and more reliable identification of missing children, helping authorities improve search and rescue operations.

CHAPTER 4
PROBLEM DEFINITION AND SCOPE

4.1 PROBLEM STATEMENT

To develop an AI-based system capable of detecting and recognizing known or missing children in real-time from live CCTV footage or video streams, using computer vision and deep learning techniques to enhance public safety and assist in child recovery efforts.

4.1.1 Goals & objectives

- To automatically detect children in video streams using detection algorithms such as MTCNN.
- To perform real-time face detection and recognition using MediaPipe and FaceNet /DeepFace frameworks.
- To create a child database containing images and identity details for recognition and comparison.
- To generate real-time alerts and notifications when a known or missing child is identified.
- To provide a user-friendly monitoring dashboard displaying detected child details and match confidence scores.
- To support law enforcement and Smart City surveillance systems by enhancing real-time monitoring and safety.

4.1.2 Statement of Scope

1. Software Description:

The proposed system is an intelligent AI-powered surveillance solution designed to detect and identify children in real-time using video input from CCTV cameras or uploaded video files. It integrates MTCNN, MediaPipe, and FaceNet/DeepFace for object detection, face extraction, and recognition respectively.

Size of Input:

Visual Data: Continuous input from live CCTV video streams or pre-recorded videos.

Database Data: Stored facial images and metadata of known children (6–7 subjects).

Bounds on Input:

Input video resolution depends on the camera's specifications (typically 720p–1080p).

Database images must be clear, front-facing, and within predefined size limits for accurate recognition.

Input Validation:

Validation ensures that each input frame and face sample meets minimum clarity and size requirements.

Invalid or low-quality images are discarded automatically before recognition.

Input Dependency:

The system's accuracy relies on proper camera placement, adequate lighting, and quality of child image datasets.

I/O State Diagram:

Input → Detection Layer (MTCNN) → Face Extraction (MediaPipe) → Recognition Layer (FaceNet/DeepFace) → Output (Alert/Result Display)

Major Inputs:

CCTV footage or uploaded videos, stored images of known children.

Major Outputs:

Detection results with bounding boxes, recognized child names, confidence percentages, and alert notifications.

2. Scope Definition:

What the Product Is and Will Do:

- The product is an AI-based detection and recognition system designed to identify known or missing children from live CCTV feeds.
- It processes real-time video streams, detects human figures, extracts faces, and compares them with the stored child database.
- It provides a visual dashboard interface showing detections, matches, and alert messages.
- It supports real-time monitoring and can integrate into Smart City surveillance frameworks.
- Ensures data security and privacy of stored images and video data.

What the Product Is Not and Will Not Do:

- The system is not a replacement for manual investigation or human supervision.
- It will not perform facial recognition for adults or unauthorized individuals outside the child dataset.
- It will not make legal or identity confirmation decisions; it serves as an aid to authorities.
- It will not share or expose personal image data to third parties without consent.

4.1.3 Business/Product Line Context:

The project falls within the domain of AI-based surveillance and safety technology aimed at supporting public safety initiatives and Smart City programs. The product is designed for law enforcement agencies, public institutions, and government bodies responsible for crowd safety and child protection. This solution can also be adapted for educational campuses, event management authorities, and transportation hubs. It leverages advanced computer vision, machine learning, and facial recognition to make surveillance more intelligent, efficient, and responsive.

4.1.4 Application Context:

The application is intended for real-time monitoring, child identification, and safety tracking in public spaces. The system can process live camera feeds or recorded videos and provide immediate recognition results.

Primary Users:

- Security officers and surveillance teams.
- Police and child protection authorities.
- Smart City control centers and monitoring staff.

Use Cases:

- Identifying missing or known children in real-time from CCTV feeds.
- Generating alerts for quick action and rescue operations.
- Maintaining a searchable record of detections for investigation.
- Supporting child safety initiatives through AI automation.
- This system aligns with the vision of proactive public safety, AI-assisted monitoring, and intelligent surveillance infrastructure for urban environments.

4.2 MAJOR CONSTRAINTS

- Data Privacy and Ethics:

  The system deals with sensitive child data, requiring strict adherence to privacy laws and ethical use of facial recognition technology.
- Hardware Limitations:

  Real-time detection and recognition require high processing power (GPU-based computation), which may limit deployment on low-resource systems.
- Lighting and Environmental Conditions:

  Poor lighting or low-resolution video can reduce accuracy in detection and face recognition.
- Database Limitations:

  The recognition accuracy heavily depends on the quality and diversity of child images in the dataset.
- Network and Storage Constraints:

  Continuous video processing demands stable network bandwidth and large storage for logs and frames.
- Model Accuracy and Optimization:

  Balancing speed and accuracy is challenging; deep models like MTCNN and FaceNet require fine-tuning and optimization for deployment.

4.3 METHODOLOGIES OF PROBLEM SOLVING AND EFFICIENCY ISSUES

The problem of real-time child detection and recognition can be approached using different methodologies, each with distinct efficiency and performance trade-offs.

1. Direct Frame Processing with Fixed Detection Models

Approach**:** Use a pre-trained MTCNN model to continuously process frames and detect human or child figures.
Performance Parameters:

- Efficiency: High frame rate, suitable for real-time processing.
- Accuracy: Limited by fixed thresholds and may misclassify small children as adults.
- Resource Use: High GPU dependency.

Efficiency Issues: Real-time frame processing can be computationally expensive; false positives may occur in crowded or complex scenes.

2. Hybrid Detection and Recognition Pipeline

Approach**:** Combine MTCNN for detection, MediaPipe for face localization, and FaceNet/DeepFace for recognition.
Performance Parameters:

- Efficiency: Balanced performance using optimized frame handling.
- Accuracy: High, due to multi-stage detection and verification.
- Scalability: Suitable for multi-camera setups.

Efficiency Issues: Requires synchronization between modules; latency may increase slightly due to sequential processing.

3. Event-Triggered Recognition and Frame Sampling

Approach: Instead of analyzing every frame, the system processes only key frames (sampled at intervals or triggered by motion detection).
Performance Parameters:

- Efficiency: Reduces computational load and improves throughput.
- Accuracy: Moderate, may miss detections in skipped frames.
- Power Efficiency: Ideal for low-resource systems.

Efficiency Issues: Frame sampling can miss short-duration appearances or fast movements.

4.3.1 SCENARIOS IN WHICH MULTI-CORE, EMBEDDED AND DISTRIBUTED COMPUTING USED

Embedded Computing

- Scenario: Edge-based AI Camera Modules
- Application: AI-powered CCTV cameras process detection locally using embedded GPUs.
- Benefits: Reduces dependency on cloud servers and improves real-time responsiveness.
- Efficiency: Low latency and high-speed performance suitable for field deployment.

Multi-Core Computing

- Scenario: Parallel Frame Processing in Surveillance Servers
- Application: Multiple frames from different cameras processed in parallel threads or cores.
- Benefits: Increases throughput and detection speed.
- Efficiency: Enables simultaneous recognition tasks without performance degradation.

Distributed Computing

- Scenario: Cloud-Based Monitoring System
- Application: Multiple camera feeds transmitted to cloud servers for processing and storage.
- Benefits: Scalable, allows multi-location deployment and centralized alert management.
- Efficiency: Supports high-volume data handling and model updates remotely.

4.4 OUTCOME

1. Real-Time Child Detection and Recognition:

   The system can accurately detect and identify children in live video feeds using AI algorithms.
2. Enhanced Public Safety:

   Provides authorities with real-time alerts and helps locate missing or known children quickly.
3. Improved Monitoring Efficiency:

   Reduces manual surveillance efforts by automating detection and recognition tasks.
4. Smart City Integration:

The solution can be integrated into existing smart surveillance systems to enhance urban safety.

5. Data Privacy and Security:

   Ensures secure storage and ethical handling of child identity data.

6. Scalability and Future Expansion:

   The architecture supports additional modules, like behavioral tracking or face aging analysis, for advanced safety solutions.

4.5 APPLICATIONS

1. Public Safety and Law Enforcement:

   Used by police and security agencies to locate missing or abducted children in real- time through CCTV networks.

2. Smart City Surveillance Systems:

   Integrated into smart city control centers to monitor crowded public areas efficiently.

3. Educational Institutions:
   Schools can use the system to monitor student presence and safety within campus areas.

4. Transportation Hubs:

   Deployed in railway stations, bus terminals, and airports to identify unaccompanied or missing children.

5. Event and Crowd Management:

   Applied during large gatherings, fairs, or festivals for real-time child tracking.

6. NGOs and Child Protection Agencies:

   Enables organizations to assist law enforcement with rapid identification during rescue missions.

7. Research and Development:

   The collected data (in anonymized form) can support further research on AI-driven safety solutions and real-time video analytics.

4.6 HARDWARE RESOURCES REQUIRED

Sr. no.	Parameter	Minimum Requirement	Justification
1	Biometric Sensors	Heart rate sensor (e.g., MAX30100) and skin conductance sensor (e.g., EDA sensor)	For detecting physiological indicators of stress, anxiety, and depression in real-time.
2	Gyroscope and Accelerometer	IMU sensor (e.g., MPU6050 or MPU9250)	To capture motion data, such as activity level, which can correlate with mental state changes.
3	Microcontroller	Arduino Nano or ESP32	To process sensor data, manage power, and communicate with mobile application.
4	Bluetooth Module	BLE module (e.g., HM-10 or ESP32)	For efficient and low-power wireless data transmission to the connected app.
5	Battery	Lithium-ion rechargeable battery (3.7V, 1000mAh)	Provides portable power supply for continuous use and user convenience.
6	Housing and Wrist strap	Lightweight, durable case and adjustable strap	Ensures comfort and wearability for prolonged periods.

Table 1: Hardware Requiremen

4.7 SOFTWARE RESOURCES REQUIRED

1. Operating System - Ubuntu/Windows/Raspbian (for Raspberry Pi)
2. Programming Languages - Python (Primary), JavaScript (for web integration)
3. Deep Learning Frameworks - TensorFlow, PyTorch
4. Computer Vision Libraries - OpenCV, Mediapipe
5. Face Detection Model – MTCNN (Multi-task Cascaded Convolutional Neural Network)
6. Face Recognition Library - FaceNet or DeepFace for embedding-based recognition
7. Database MongoDB/Firebase / MySQL for storing user and facial data
8. API Framework - Flask /FastAPI / Node.js for backend communication
9. Frontend Development - React.js / HTML5/Bootstrap for web-based dashboard
10. Cloud and Hosting Services - AWS, Google Cloud, or Azure for scalability
11. Development Environment - Visual Studio Code / Jupyter Notebook
12. Version Control - Git and GitHub for collaborative code management

CHAPTER 5
PROJECT PLAN

5.1 PROJECT ESTIMATES

5.1.1 Reconciled Estimates

5.1.1.1 Cost Estimates:

Components	Price (INR)	Quantity
High-Resolution CCTV Camera	1500–2500	2
USB Webcam (for local testing)	750-1200	1
Monitor / Display	4000-6000	1
Power Adapter + Cables	500-800	1
Speaker / Buzzer (for alerts)	200-500	1
Software Tools (Open Source - Python, YOLOv8, FaceNet, MediaPipe)	free	-

Table 2: Cost Estimates

5.1.1.2 Time Estimates:

Time estimation in hours

Total time required: 48-96 hours.

Process	Time in Minutes
Image/Video Capture	Continuous (real-time)
Face Localization using MediaPipe	Less than 1
Feature Extraction using FaceNet	About 2 seconds 1
Database Matching and Alert Generation	Less than 1

Table 3: Time Estimates

5.1.2 Project Resources

Hardware Resources:

1. High-Resolution CCTV Camera or IP Camera – For capturing real-time video footage in public areas.
2. GPU-Enabled Computer (e.g., NVIDIA RTX Series) – For training and running deep learning models efficiently.
3. Raspberry Pi or Jetson Nano (optional) – For edge-based AI processing and deployment in real- time systems.
4. Storage Device (SSD or HDD) – For storing training datasets and processed video frames.
5. Power Supply / Adapter – To ensure stable operation of the hardware setup.
6. Wi-Fi or Ethernet Module – For network communication and data transfer.
7. Monitor / Display Unit – To visualize live detection and alerts.
8. Cables and Connectors – For interconnecting components and ensuring data flow.
9. Webcam / USB Camera (optional) – For testing and prototype demonstrations.

Software Resources:

1. Python 3.x – Primary programming language for model development.
2. MediaPipe – For face localization and landmark detection.
3. FaceNet / DeepFace Library – For facial recognition and embedding generation.
4. OpenCV – For image and video processing operations.
5. TensorFlow / PyTorch – For deep learning model implementation and training.
6. Jupyter Notebook / Google Colab – For experimentation and model evaluation.
7. MySQL / SQLite Database – For storing known child profiles and embeddings.
8. Visual Studio Code / PyCharm – Integrated Development Environments (IDE) for coding and debugging.
9. GitHub / Git – For version control and collaborative project management.
10. Windows / Linux Operating System – For model execution and software compatibility.
11. Alert System Integration Tools (e.g., Twilio or SMTP Email API) – For sending alerts when a child is identified.

5.2 RISK MANAGEMENT W.R.T. NP – HARD ANALYSIS

5.2.1 Risk Identification

1. Have top software and customer managers formally committed to support the project? Answer: No
2. Are end-users enthusiastically committed to the project and the system/product to be built? Answer: Yes

3. Are requirements fully understood by the software engineering team and its customers? Answer: Yes
4. Have customers been involved fully in the definition of requirements? Answer: Yes
5. Do end-users have realistic expectations? Answer: Yes
6. Does the software engineering team have the right mix of skills? Answer: Yes
7. Are project requirements stable?
   Answer: No, depends upon the hardware used.
8. Is the number of people on the project team adequate to do the job? Answer: Yes
9. Do all customer/user constituencies agree on the importance of the project and on the requirements for the system/product to be built?
   Answer: Yes

5.2.2 Risk Analysis

Risk Description	Probability	Impact on Schedule	Impact on Quality	Impact on Overall
Inaccurate Face Detection in Crowded Areas	Moderate	Moderate	High	High
Poor Lighting or Camera Quality Issues	Moderate	Low	High	Moderate
Insufficient Dataset Diversity	Moderate	Moderate	High	High
Hardware or Camera Malfunction	Low	Moderate	Moderate	Moderate
Real-time Processing Delays	Moderate	High	Moderate	High
Network or Connectivity Issues	Low	Moderate	Moderate	Moderate
Integration Issues Between MTCNN, MediaPipe, and FaceNe	Moderate	Moderate	High	High
Data Privacy and Security Concerns	Low	Low	High	High
Misidentification or False Alerts	Moderate	Moderate	High	High

System Maintenance and Scalability Issues	Moderate	High	Moderate	High

Table 4: Risk Table

Probability	**Value**	**Description**
High	Probability of occurrence is	>75%
Moderate	Probability of occurrence is	26 – 75%
Low	Probability of occurrence is	<25%

Table 5: Risk Probability Definitions

Impact	**Value**	**Risk Description**
Low	< 30%	Poor lighting or camera quality issues
Moderate	30-60%	Inaccurate face detection in crowded areas
Moderate	30-60%	Insufficient dataset diversity
Low	< 30%	Network or connectivity issues
Moderate	30-60%	Integration issues between MTCNN, MediaPipe, and FaceNet
Moderate	30-60%	Real-time processing delays
Moderate	30-60%	Misidentification or false alerts
Low	< 30%	System maintenance and scalability issues
Low	< 30%	Data privacy and security concerns
Low	< 30%	Hardware or camera malfunction

Table 6: Risk Impact Definitions

5.2.3 Overview of Risk Mitigation, Monitoring, Management

Risk ID	R1
Risk Description	Insufficient or poor-quality CCTV footage
Category	Data Quality / Environment
Source	Low-resolution cameras, poor lighting or occlusion
Probability	Medium
Impact	High
Response	Mitigate
Strategy	Use preprocessing, face enhancement, and multiple video sources
Risk Status	Occurred / Possible

Table 7: Risk Delay in Component Procurement

Risk ID	R2
Risk Description	Incorrect child recognition due to small dataset
Category	Requirements / Dataset
Source	Limited identity samples
Probability	Low
Impact	High
Response	Mitigate
Strategy	Use synthetic-data augmentation (AI generated), continuously update DB
Risk Status	Identified

Table 8: Risk in Inadequate Sensor Accuracy

5.3 PROJECT SCHEDULE

5.3.1 Major Tasks in Project Stages

- Task 1: Literature Survey & Requirement Analysis
- Task 2: Dataset Collection & Preprocessing
- Task 3: Model Development – MTCNN + Mediapipe + FaceNet
- Task 4: System Integration & Testing
- Task 5: Deployment & Documentation

5.3.2 Task Network

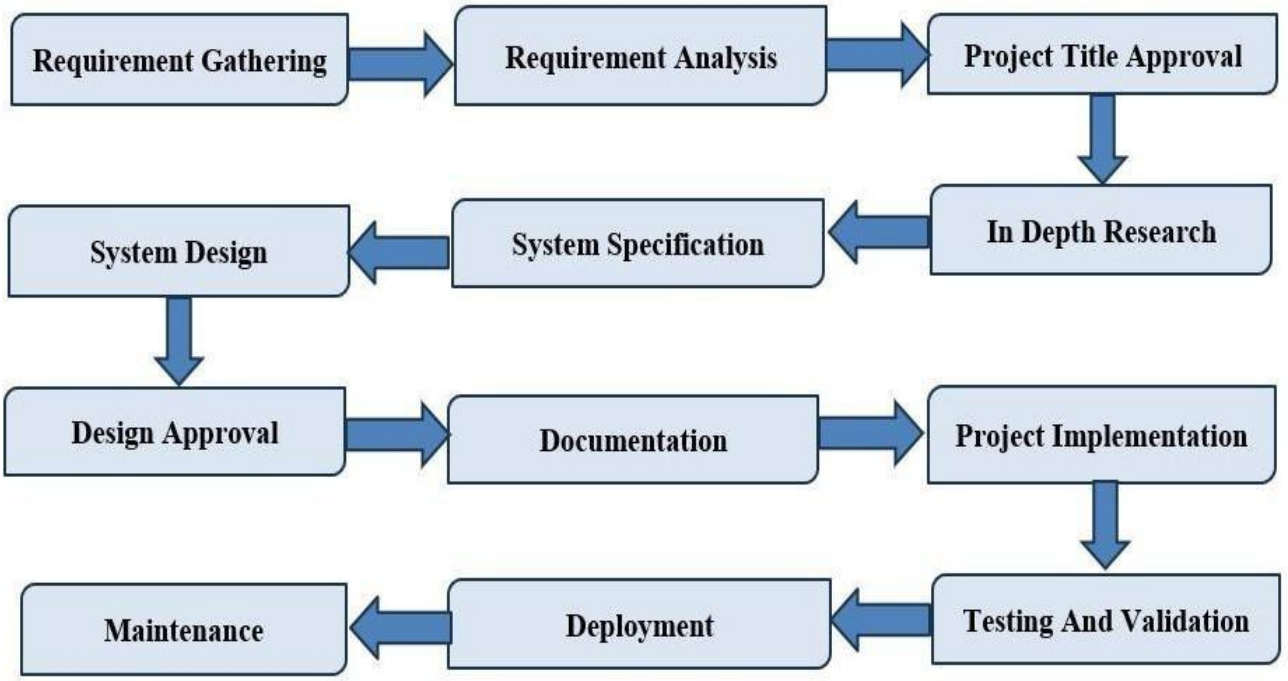


Figure 1: Task Network Flow

5.3.3 Timeline Chart

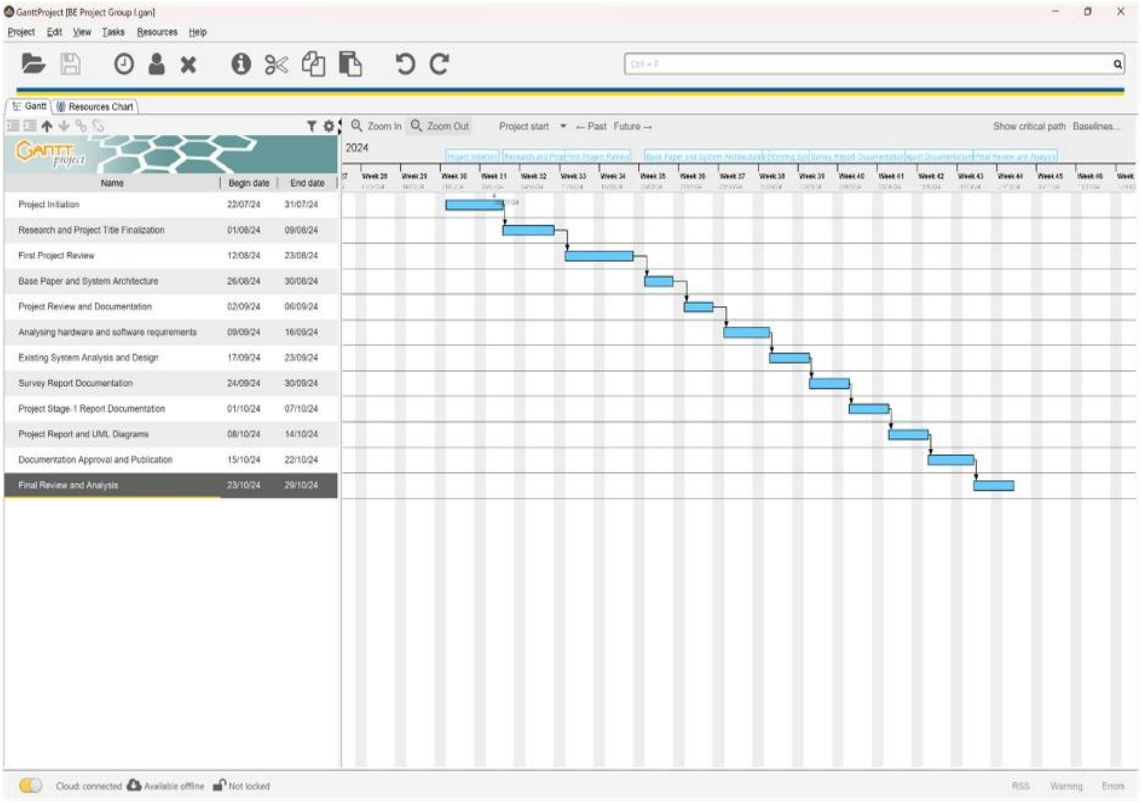


Figure 2: Timeline as per Gantt Chart

5.4 TEAM ORGANISATION

5.4.1 Team Structure

Project Manager: Janhavi Mahajan and Gayatri Kadam
System Developer 1: Janhavi Mahajan
System Developer 2: Gayatri Kadam
System Developer 3: Tanisha Kahane
System Developer 4: Isha Soni
Analyst: Janhavi Mahajan Gayatri Kadam , Tanisha Kahane , Isha Soni
Expense Manager: Janhavi Mahajan and Gayatri Kadam
Documentation: Tanisha Kahane and Isha Soni

CHAPTER 6

SOFTWARE REQUIREMENT SPECIFICATION

6.1 INTRODUCTION

6.1.1 Purpose and Scope of Doc

Purpose:

This document serves as a comprehensive guide and reference for stakeholders involved in the AI-based Lost Child Identification project. The primary purpose of this document is to outline the objectives, system design, architecture, algorithms, and implementation plan for developing an intelligent surveillance system that automates child face detection and recognition in real-time. The system uses MTCNN (Multi-task Cascaded Convolutional Neural Network) for accurate face detection and landmark extraction, along with FaceNet/DeepFace for identification. It aims to assist parents, authorities, and public safety personnel in locating missing or lost children efficiently using advanced computer vision and deep learning technologies.

Scope:

The scope of this document includes an overview of project objectives, system architecture, hardware and software requirements, data acquisition and preprocessing methods, training and testing methodologies, and performance evaluation. It also covers the use of MTCNN for face detection, MediaPipe for face alignment and cropping, and FaceNet/DeepFace for recognition. Ethical considerations such as privacy and data security are also addressed. The document further includes testing and validation processes, user interface design, and potential future enhancements. The system is intended for deployment in public areas such as railway stations, malls, airports, and parks for real-time face-based child identification and alert generation.

6.1.2 Overview of Responsibilities

1. Designing the software architecture and integrating AI models (MTCNN, MediaPipe, FaceNet).
2. Developing algorithms for real-time child detection and face recognition.
3. Building the database for storing and matching known child profiles.
4. Integrating camera input with the detection and recognition system.
5. Testing, debugging, and validating the detection accuracy and system responsiveness.
6. Ensuring efficient hardware and software integration for real-time operation.
7. Writing documentation for system setup, usage, and maintenance.
8. Managing and updating the system with new data for continuous improvement.

6.2 USAGE SCENARIO

This section demonstrates practical situations illustrating how the AI For Lost Child Identification functions in real-world scenarios.

Scenario 1: Child Detection in a Crowded Mall

A parent reports their child missing in a busy shopping mall. The mall's surveillance system, integrated with the AI-based software, immediately activates child search mode. Using MTCNN, the system scans live CCTV feeds to detect faces in real time. MediaPipe then refines and crops the facial regions. Each extracted face is processed using FaceNet to generate embeddings and compared with the pre-registered child database. When a match is found, the system displays the child's details and location on the control screen, alerting mall security for immediate action.

Scenario 2: Real-Time Identification at a Railway Station

At a busy railway station, the system continuously monitors passengers. When a lost child appears in a camera's view, MTCNN detects the face and extracts key facial landmarks. MediaPipe captures the facial region, and FaceNet compares it with the missing child database. Upon a successful match, an alert with the child's name, similarity score, and camera ID is sent to the control room, helping authorities locate and assist the child promptly.

Scenario 3: Airport Deployment for Missing Child Tracking

In an airport deployment, the system monitors multiple camera feeds. If an announcement is made about a missing child, the system instantly begins analyzing live video streams. MTCNN detects faces, which are then processed and matched against the airport's security database using FaceNet. Once a match is identified, the system notifies security staff with relevant details, ensuring quick response and enhanced public safety.

6.2.1 USER PROFILES

User 1: Security Operator

- Name: Rajesh
- Age: 35
- Gender: Male
- Background: Rajesh works as a surveillance control officer at a metropolitan railway station. His primary duty is to monitor live CCTV feeds and respond to security alerts. The AI system assists him by automating child detection and recognition, allowing faster response in locating missing children.

User Profile Summary:
Rajesh relies on the system to monitor multiple cameras simultaneously without manual fatigue. The AI- based tool provides alerts with precise details, improving efficiency and accuracy in responding to emergencies and supporting child safety operations.

User 2: Parent or Guardian

- Name: Meera Sharma
- Age: 40
- Gender: Female
- Background: Meera is a parent who frequently visits public places with her young child. She can register her child's facial data in the system's database through a secure portal. In case her child goes missing, she can quickly alert authorities, enabling the AI system to begin the automated search immediately.

Summary:
Meera values the system's reliability and speed in responding to emergencies. The system provides her peace of mind, knowing that an intelligent, real-time detection mechanism is actively searching for her child across public surveillance networks.

6.2.2 Use Cases

Sr. No.	Use case	Description	Actors	Assumptions
1	Detect Child	The system detects children or persons in a live CCTV feed using MTCNN detection model. Detected bounding boxes are passed for further face extraction..	CCTV Camera (Primary actor) System (Secondary actor)	CCTV feed is active and stable. Camera resolution ≥ 720p. MTCNN model is pre- trained and loaded successfully
2	Extract Faces	Once a child/person is detected, Mediapipe extracts the facial region from the bounding box for recognition.	User (Primary actor) System	Detected faces are visible and not occluded. Lighting conditions allow proper extraction.
3	Recognize Child Face	The extracted face is passed to FaceNet, which generates embeddings and compares them with the known child database to find a match	1. System (Primary actor)	Database of registered children exists. FaceNet embeddings are precomputed for all known faces.
4	Display Identification Result	The system displays the identified child's name, confidence score, and location of detection on the dashboard.	User (Primary actor) System	Database connection is active. The user has dashboard access rights.

Table 9: Use Case

6.2.3 Use Case View

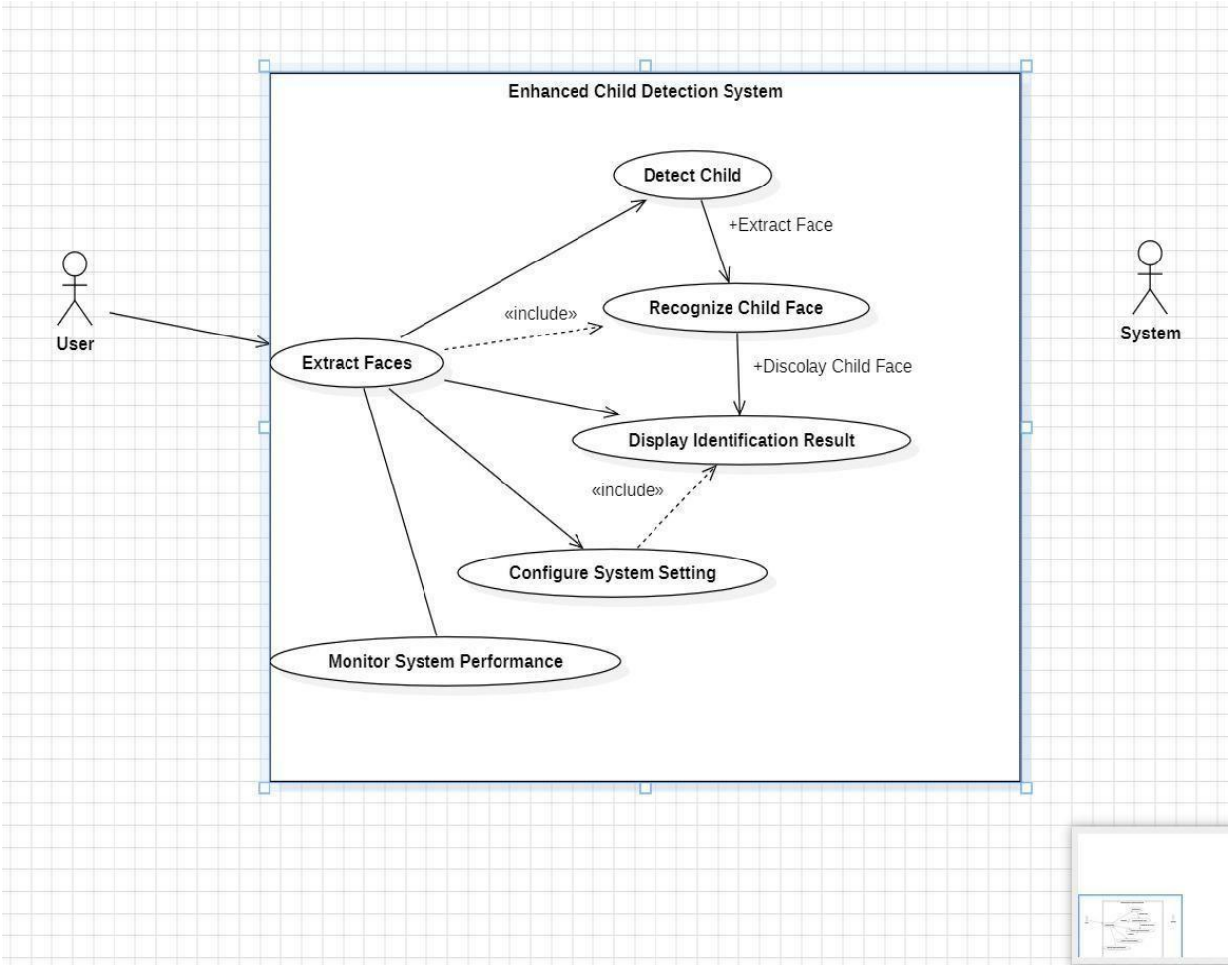


Figure 3 : Use Case Diagram

6.3 DATA MODELAND DESCRIPTION

6.3.1 Data Description

Our project uses a custom dataset. We aim to generate our own dataset by performing testing on various individuals in the college premises. This involves parameters like Age, Gender, Stress Level, Phone Usage Level, Heart Rate, Blood Pressure, SpO_2, etc.

6.3.2 Data Objects and Relationships

Data objects and their major attributes and relationships among data objects are described using an ERD- like form.

6.4 FUNCTIONAL MODEL AND DESCRIPTION

6.4.1 Data Flow Diagram

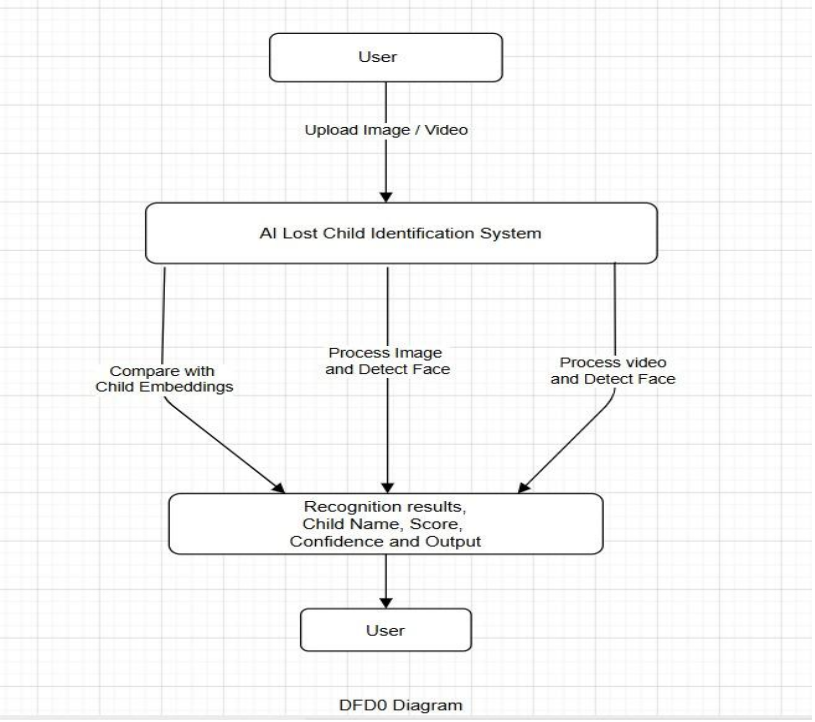


Figure 4: DFD Level 0 (Context Diagram)

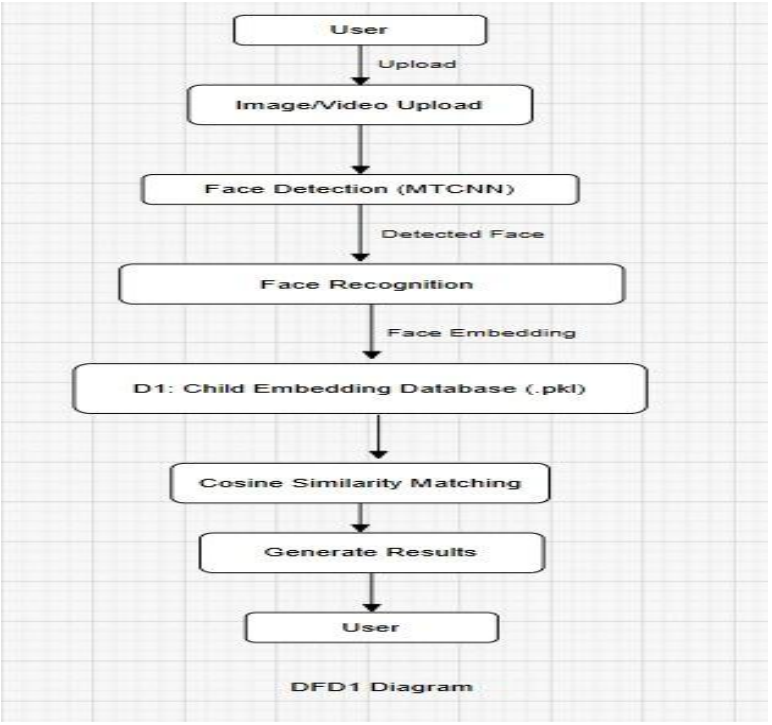


Figure 5: DFD Level 1 AI-Based Lost Child Identification System

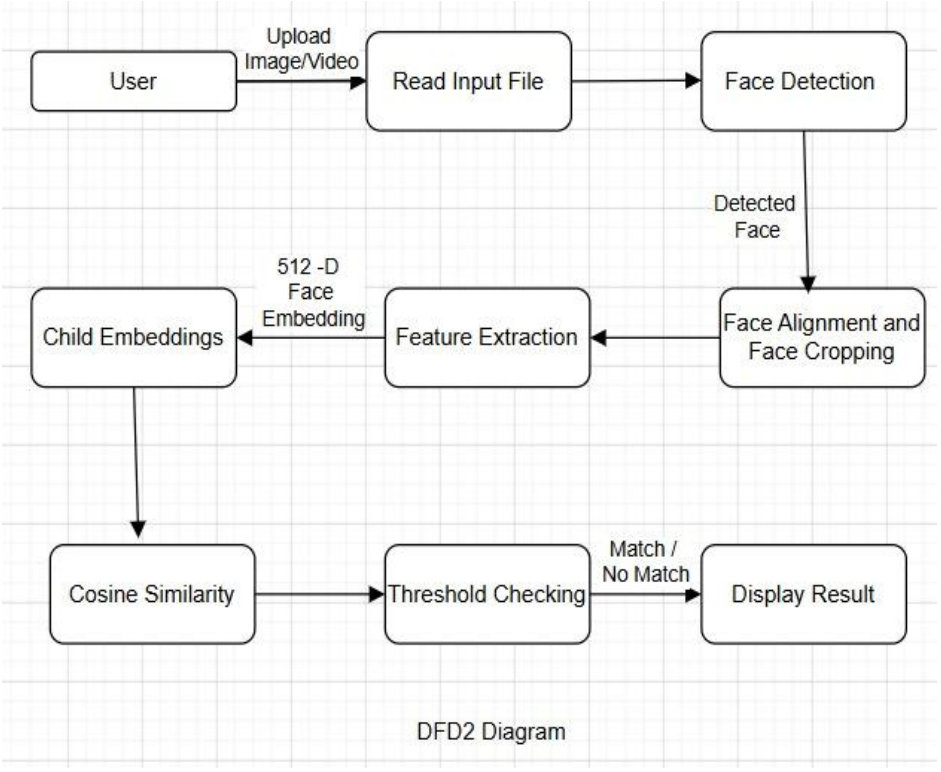


Figure 6: DFD Level 2 Face Recognition Module

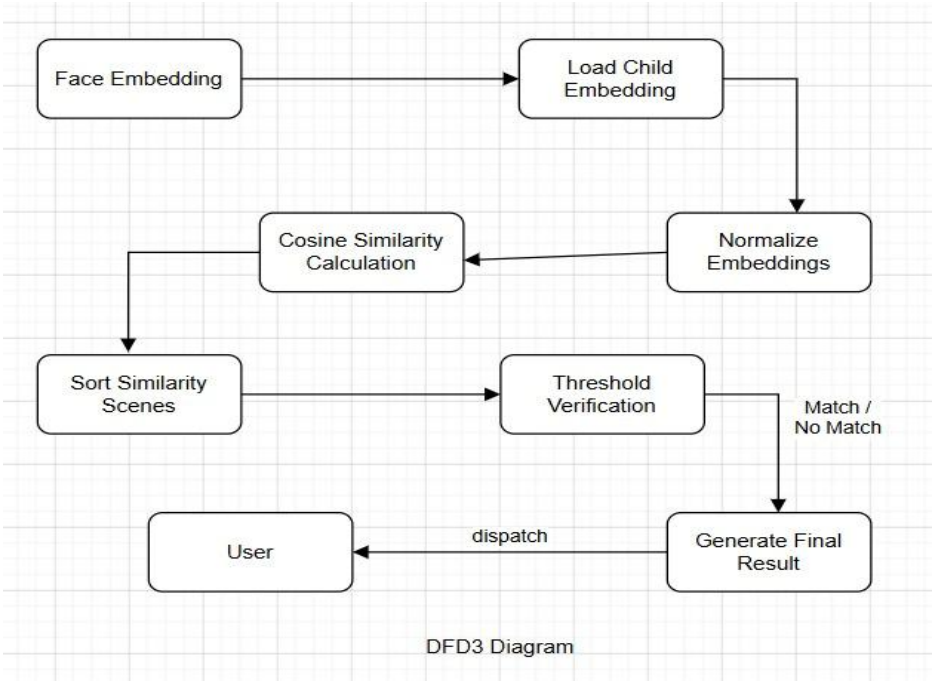


Figure 7: DFD Level 3 Similarity Matching Process

6.4.2 Activity Diagram

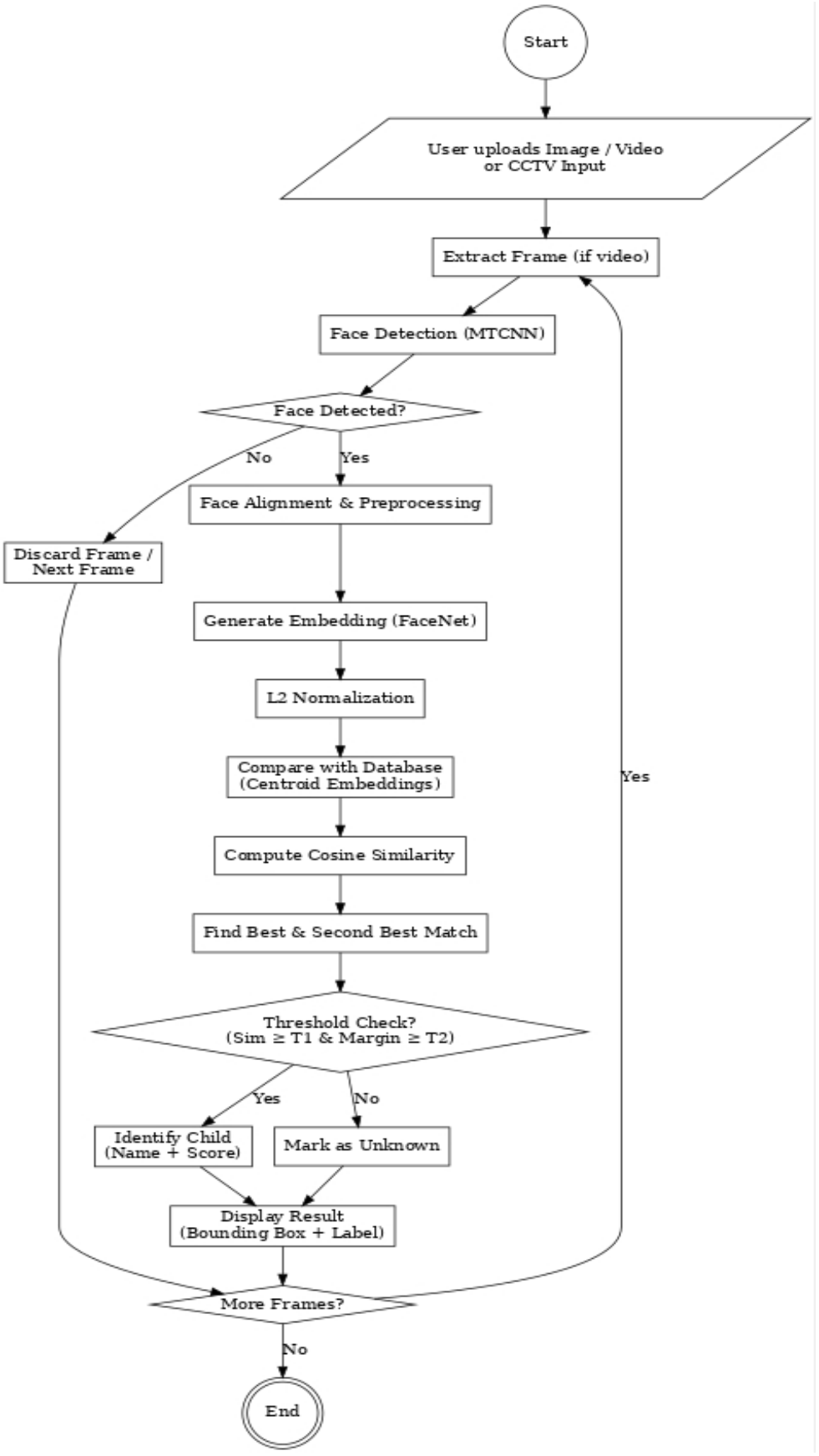


Figure 8 : Activity Diagram

6.4.3 State Diagram

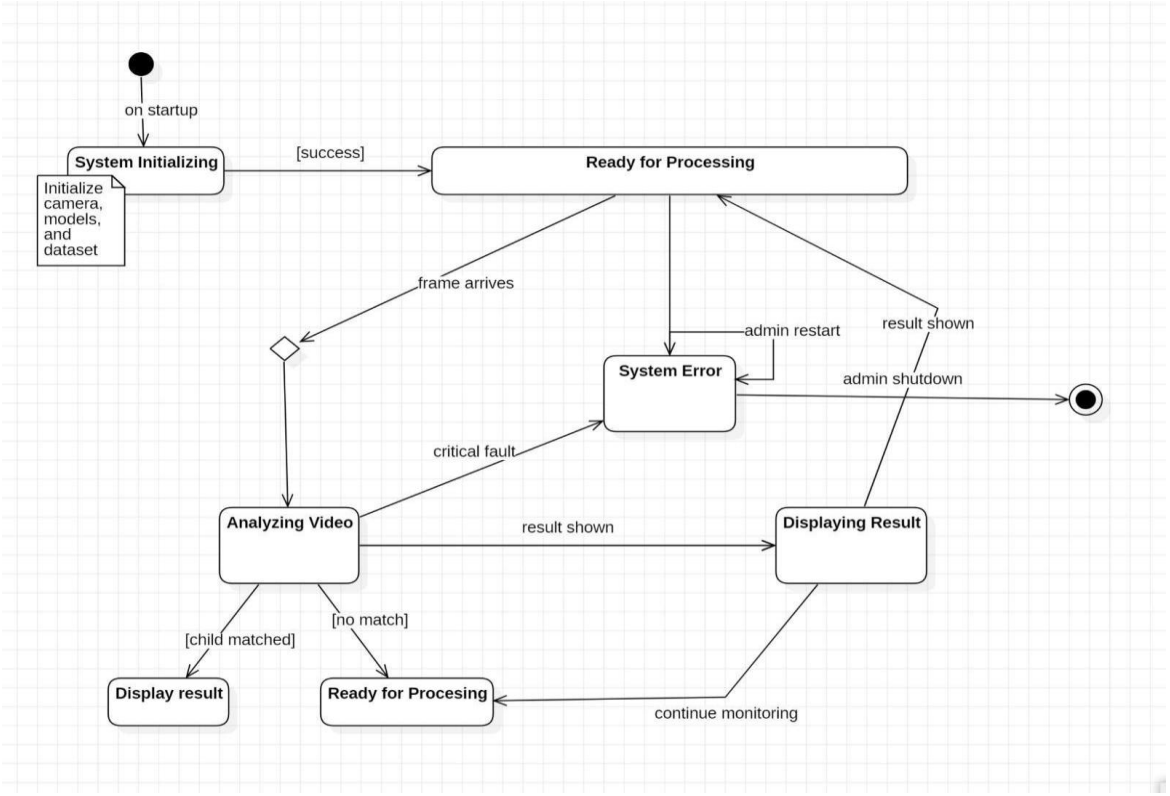


Figure 9 : State Diagram

6.4.4 Design Constraints

- Data Privacy and Security
- Camera Accuracy and Reliability
- Processing Speed and Real-time Detection
- System Hardware Limitations
- Network Connectivity
- Database Management
- Model Limitations
- Scalability and Compatibility

6.4.5 Software Interface Description

1. CCTV Camera Interface

- Video Feed Input: CCTV cameras continuously capture live video streams of public areas.
- Frame Extraction: The system extracts video frames at a fixed rate (e.g., 10 FPS) for analysis.
- Data Transmission: Frames are transmitted securely to the processing module for detection.
- Connectivity: Supports IP-based camera streaming through RTSP or HTTP protocols.

2. Detection and Recognition Interface

- Face Detection (MTCNN): Detects human faces within video frames and identifies facial regions of interest (ROI) along with key landmarks (eyes, nose, mouth).
- Face Extraction (Mediapipe): Extracts facial regions from detected individuals for further analysis.
- Face Recognition (FaceNet): Generates embeddings for each face and compares them to the known child database.
- Confidence Score Calculation: Each match is assigned a confidence score to evaluate reliability.

3. Database Interface

- Data Storage: Stores pre-registered child details such as name, photo embeddings, and identification number.
- Query Operations: Enables fast search and comparison between new detections and stored profiles.
- Data Security: Implements encryption and authentication to prevent unauthorized data access.
- Update & Maintenance: Allows administrators to add or update child data records easily.

4. Dashboard Interface

- Detection Visualization: Displays live detections, matched faces, and their confidence levels on the user interface.
- Location Tracking: Shows the camera location where a child was identified.
- User Notifications: Sends alerts or notifications to authorities or parents if a match is confirmed.
- Access Control: Provides different login modes for Admin and Operator users.

5. Admin Control Interface

- System Configuration: Allows the admin to modify system thresholds (detection confidence, camera selection, etc.).
- User Management: Admin can create, edit, or remove user accounts with role-based

access.

- Report Generation: Generates daily/weekly logs of detections and matches.
- Performance Monitoring: Displays FPS rate, system load, and detection latency.

6. Security Interface

- Encryption: All communication between cameras, servers, and dashboards is encrypted using SSL/TLS.
- Authentication: Only authorized users can log into the system dashboard.
- Audit Trail: Maintains logs of all user actions for accountability and traceability.

CHAPTER 7

DETAILED DESIGN DOCUMENT USING APPENDIX A AND B

7.1 INTRODUCTION

This document specifies the design that is used to solve the problem of real-time child detection and identification in public spaces. The system employs artificial intelligence, deep learning, and computer vision to identify lost or known children using video input from CCTV cameras. The architectural design focuses on efficient data flow between object detection, face recognition, and alert generation modules. The goal is to develop a reliable and accurate system that can assist authorities in locating children quickly and effectively through automated surveillance

7.2 ARCHITECTURAL DESIGN

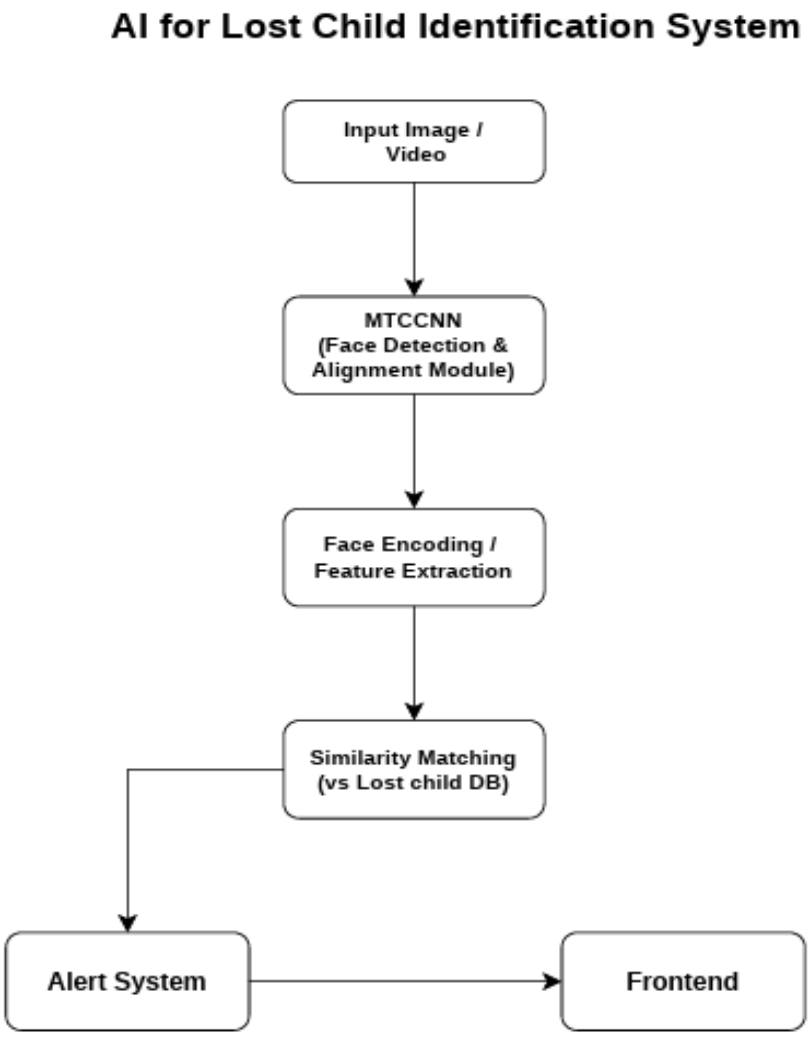


Figure 10: System Architecture Design

7.3 DATA DESIGN (USING APPENDICES A AND B)

7.3.1 Internal Software Data Structure

1. Main Control Structure:
   Detection Controller: This is the central control module that manages video input, coordinates between the detection, recognition, and alert subsystems, and ensures smooth operation of the entire AI pipeline.

2. Data Management:
   Frame Data Buffer: Temporarily stores frames extracted from live video or CCTV feeds before processing.
   Detection Results Buffer: Holds intermediate outputs such as face bounding box coordinates, facial landmarks, and confidence levels generated by MTCNN detection.
   Face Embedding Storage: Stores numerical face embedding vectors generated by the FaceNet/DeepFace model for comparison with the database.
   Alert Log Storage: Saves alert history, timestamps, and match confidence results for monitoring and reporting.

3. AI Model Interfaces:
   Face Detection Module (MTCNN): Detects human faces within video frames and identifies facial regions of interest (ROI) along with key landmarks (eyes, nose, mouth).
   Face Detection Module (MediaPipe): Locates and crops faces from detected objects for accurate recognition.
   Face Recognition Module (FaceNet/DeepFace): Converts face images into embedding vectors and performs similarity matching against stored child data.

4. Database Management:
   Child Database Interface: Manages image storage, embedding vectors, and identity data of known or missing children.
   Alert Database: Maintains records of all detections, alerts, and confidence scores for audit and tracking.

5. User Interface (Frontend Dashboard):
   Display Interface: Presents detection results, recognized child names, and confidence percentages in real-time.
   Control Interface: Allows users or security personnel to configure thresholds, update databases, and manage notifications.

6. Feedback and Control:
   Error Handling Module: Logs issues such as missing frames, poor lighting, or detection failures for debugging.
   System Control Logic: Manages the workflow between modules (MTCNN →

MediaPipe → FaceNet → Alert).

7. Configurations:
   System Configurations: Includes model weights, thresholds, frame rate settings, and hardware parameters.
   Communication Configurations: Defines paths and communication protocols between modules, APIs, and dashboard.

7.3.2 Global Data Structure

Video Frame Data: Stores sequential frames captured from CCTV or input video.
Detection Data: Contains face bounding boxes, facial landmarks, and confidence scores generated by **MTCNN**.
Face Embedding Data: Holds numerical feature vectors generated by FaceNet for each detected face.
Database Data: Stores all registered child face embeddings, IDs, and metadata.
Alert Data: Maintains logs of matches, timestamps, and system actions.
System Status: Tracks ongoing operations, alert activity, and error logs.

7.3.3 Temporary Data Structure

Frame Cache: Temporarily holds processed frames before they are sent to recognition or alert modules.
Recognition Queue: Maintains the list of faces currently being compared with the database.
Alert Queue: Stores recent matches and alert events waiting to be displayed or logged.

7.3.4 DATABASE DESCRIPTION

The project uses a custom dataset of child face images for training and recognition purposes.

- Child Dataset: Contains labeled face images of 6–7 known children, augmented with AI- generated samples for robustness.
- Database Schema:
  - Child_ID: Unique identifier for each child.
  - Child_Name: The name associated with each record.
  - Face_Embedding: 128-dimensional feature vector from FaceNet.
  - Confidence_Score: Match probability stored after each recognition event.
  - Timestamp & Location: The time and place of detection for real-time alert tracking.

The dataset will be continuously expanded to improve recognition accuracy. Data augmentation techniques such as rotation, lighting adjustments, and scaling will be used to make the model more adaptable to real-world CCTV conditions.

7.4 COMPONENT DESIGN

7.4.1 Class Diagram

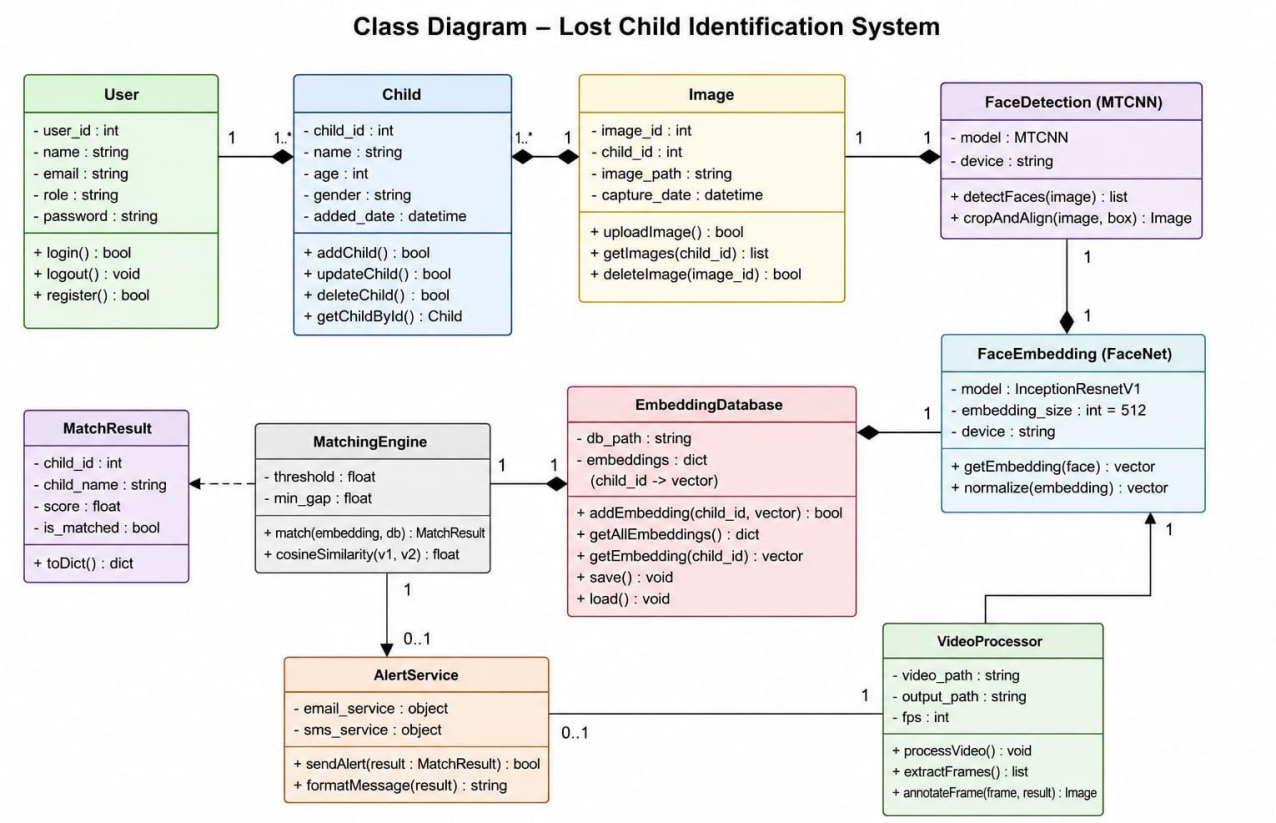


Figure 11: Class Diagram

7.4.2 Sequence Diagram:

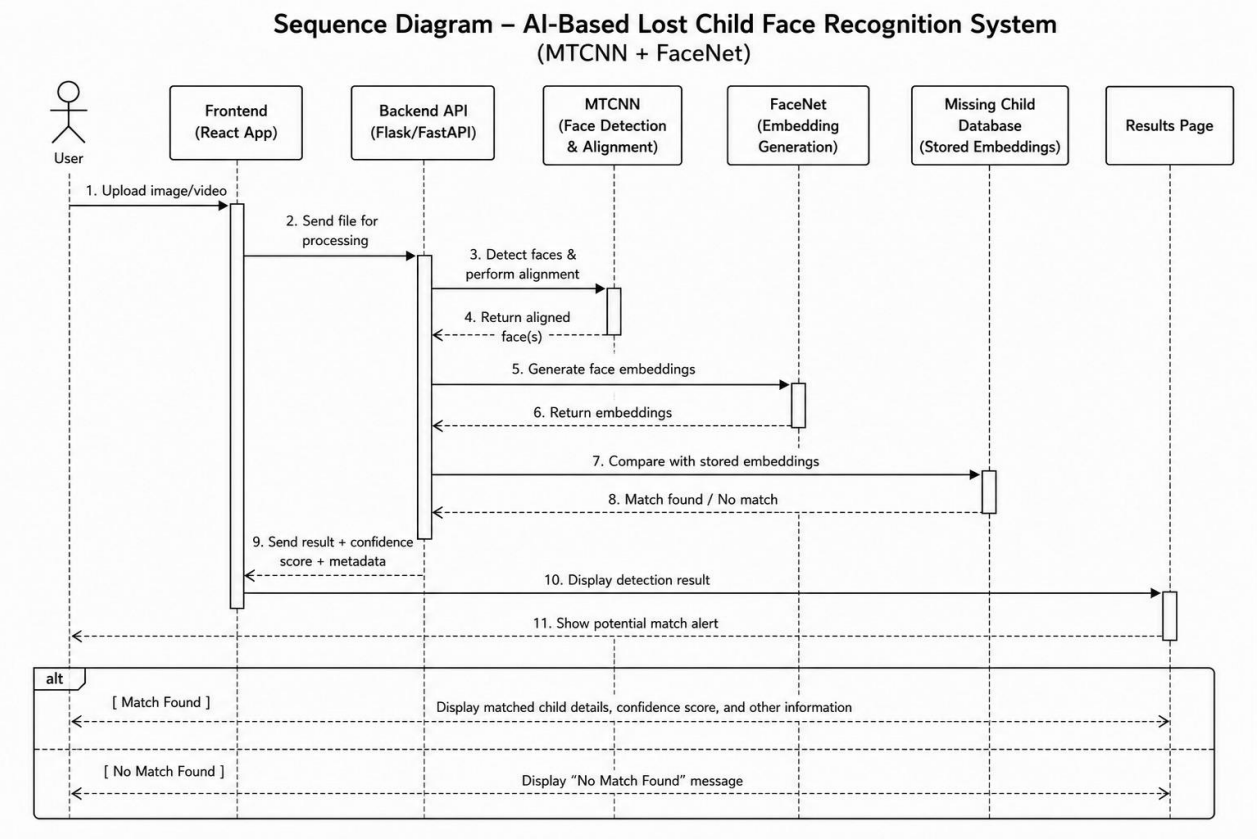


Figure 12 : Sequence Diagram

CHAPTER 8
PROJECT IMPLEMENTATION

8.1 INTRODUCTION

The Lost Child Identification System is an AI-based solution developed to assist in locating missing children through automated face recognition. The system processes images and videos, detects faces using MTCNN (Multi-task Cascaded Convolutional Network), and generates facial embeddings using FaceNet. These embeddings are compared with stored child embeddings using cosine similarity matching. The proposed system reduces manual monitoring efforts and enables faster identification of children in crowded public places such as railway stations, malls, airports, and parks. The system is implemented using Python, FastAPI, OpenCV, PyTorch, and FaceNet-PyTorch libraries.

8.2 TOOLS AND TECHNOLOGIES USED

Tool / Technology	Purpose
Python	Backend development
FastAPI	API development and communication
OpenCV	Image and video processing
PyTorch	Deep learning framework
MTCNN	Face detection and face alignment
FaceNet	Feature extraction and embedding generation
NumPy	Numerical computations
Pillow (PIL)	Image handling
Uvicorn	FastAPI server execution
VS Code	Development environment
GitHub	Version control and project hosting

8.3 METHODOLOGIES/ALGORITHM DETAILS

The system follows a face recognition pipeline consisting of face detection, feature extraction, and similarity matching.

Methodology Steps

1. Input image or video is uploaded.
2. MTCNN detects and aligns faces.
3. FaceNet extracts 512-dimensional embeddings.
4. Embeddings are normalized.

5. Cosine similarity is calculated with stored child embeddings.
6. Best matching child is identified.
7. Similarity score and confidence level are displayed.

8.3.1 Algorithm 1/Pseudo Code

```
import logging
import shutil
import tempfile
from pathlib import Path
import math
import numpy as np
import json
from fastapi import FastAPI, UploadFile, File, HTTPException
from fastapi.encoders import jsonable_encoder
from fastapi.responses import JSONResponse
from fastapi.middleware.cors import CORSMiddleware
from fastapi.staticfiles import StaticFiles
from . import image_face_service
from . import video_face_service

# --- Basic Configuration ---
logging.basicConfig(level=logging.INFO)
logger = logging.getLogger(__name__)

# --- FastAPI App Initialization ---
app = FastAPI(title="Face Recognition API")

# --- CORS Configuration ---
origins = [
    "http://localhost:3000",
    "http://localhost:5173",
    "http://127.0.0.1:3000",
    # Add your frontend domain here when you deploy
]

app.add_middleware(
    CORSMiddleware,
    allow_origins=origins,
    allow_credentials=True,
    allow_methods=["*"],
    allow_headers=["*"],
)

# --- Static File Serving ---
```

```
BASE_DIR = Path(__file__).resolve().parent
STATIC_DIR = BASE_DIR / "static"
OUTPUT_IMAGES_DIR = STATIC_DIR / "outputs" / "images"
OUTPUT_VIDEOS_DIR = STATIC_DIR / "outputs" / "videos"

# Create output directories if they don't exist
OUTPUT_IMAGES_DIR.mkdir(parents=True, exist_ok=True)
OUTPUT_VIDEOS_DIR.mkdir(parents=True, exist_ok=True)

# Mount the static directory to serve files
app.mount("/static", StaticFiles(directory=STATIC_DIR), name="static")

# --- API Endpoints ---

@app.get("/api/health")
def health():
    """Health check endpoint."""
    return {"status": "ok"}

@app.post("/face/recognize/image")
async def recognize_face_image(file: UploadFile = File(...)):
    def _sanitize_for_json(obj):
        if isinstance(obj, (float, np.floating)):
            val = float(obj)
            return val if math.isfinite(val) else None
        if isinstance(obj, (int, np.integer)):
            return int(obj)
        if isinstance(obj, np.ndarray):
            return _sanitize_for_json(obj.tolist())
        if isinstance(obj, set):
            return _sanitize_for_json(list(obj))
        if isinstance(obj, dict):
            return {k: _sanitize_for_json(v) for k, v in obj.items()}
        if isinstance(obj, list):
            return [_sanitize_for_json(v) for v in obj]
        if isinstance(obj, tuple):
            return [_sanitize_for_json(v) for v in obj]
        return obj

    def _safe_json_response(payload):
        encoded = jsonable_encoder(payload)
        encoded = _sanitize_for_json(encoded)
        try:
            json.dumps(encoded, allow_nan=False)
        except ValueError:
                encoded = {"success": False, "message": "Response contained non-JSON-compliant
```

```
values."}
        return JSONResponse(content=encoded, status_code=200)

    if not (file.content_type or "").startswith("image/"):
        return _safe_json_response({"success": False, "message": "Invalid file type. Please upload an
image."})

    try:
        image_bytes = await file.read()
    except Exception as e:
        return _safe_json_response({"success": False, "message": f"Failed to read uploaded file:
{e}"})
    finally:
        try:
            file.file.close()
        except Exception:
            pass

    try:
processed_b64,payload=
image_face_service.detect_and_recognize_faces_from_bytes(image_bytes)
        if isinstance(payload, dict):
            payload.setdefault("processed_image_base64", processed_b64)
        return _safe_json_response(payload)
    except Exception as e:
        logger.exception("Error during image-face detection")
        return _safe_json_response({"success": False, "message": f"Image face detection failed: {e}"})

@app.post("/api/detect/image-face")
async def detect_image_face(file: UploadFile = File(...)):
    return await recognize_face_image(file)

@app.post("/api/detect/image")
async def detect_image_compat(file: UploadFile = File(...)):
    return await recognize_face_image(file)

@app.post("/face/recognize/video")
async def recognize_face_video(file: UploadFile = File(...)):
    content_type = (file.content_type or "").lower()
    suffix = Path(file.filename or "").suffix.lower()
    is_video = content_type.startswith("video/")
    is_octet_stream = content_type in {"application/octet-stream", "binary/octet-stream"}
    is_video_ext = suffix in {".mp4", ".mov", ".avi", ".mkv", ".webm", ".m4v"}
    if not (is_video or (is_octet_stream and is_video_ext)):
        raise HTTPException(
            status_code=400,
```

```
            detail=f"Invalid file type. Expected a video. content_type={file.content_type}, filename={file.filename}",
        )

    tmp_path = None
    try:
            logger.info("Video upload received: filename=%s content_type=%s", file.filename, file.content_type)
            with tempfile.NamedTemporaryFile(delete=False, suffix=Path(file.filename).suffix) as tmp_file:
            shutil.copyfileobj(file.file, tmp_file)
            tmp_path = Path(tmp_file.name)

        logger.info("Saved uploaded video to temp path: %s", str(tmp_path))

        meta, out_path = video_face_service.recognize_video(
            tmp_path,
            output_dir=OUTPUT_VIDEOS_DIR,
        )

        logger.info("Video processing complete. Output: %s", str(out_path))

        base_url = "http://localhost:8000"
        output_url = f"{base_url}/static/outputs/videos/{out_path.name}"

        return {"success": True, "output_video_url": output_url, "meta": meta}
    except Exception as e:
        logger.exception("Error during video face recognition")
        raise HTTPException(status_code=500, detail=f"Video face recognition failed: {e}")
    finally:
        if tmp_path is not None and isinstance(tmp_path, Path) and tmp_path.exists():
            try:
                tmp_path.unlink()
            except Exception:
                pass
        try:
            file.file.close()
        except Exception:
            pass

@app.post("/api/detect/video-face")
async def detect_video_face(file: UploadFile = File(...)):
    return await recognize_face_video(file)

@app.post("/api/detect/video")
async def detect_video_compat(file: UploadFile = File(...)):
```

```
    return await recognize_face_video(file)

if __name__ == "__main__":
    import uvicorn
    # This block allows running the app directly for development
    # Use the command 'uvicorn app.main:app --reload' for production-like Gunicorn/Uvicorn setup
    uvicorn.run(app, host="0.0.0.0", port=8000)
```

8.3.2 Algorithm 2/Pseudo Code

```
import logging
import math
import tempfile
from pathlib import Path
from typing import Dict, List, Tuple

import cv2
import numpy as np

from . import face_service

logger = logging.getLogger(__name__)

def _clamp_box(x1: int, y1: int, x2: int, y2: int, w: int, h: int) -> Tuple[int, int, int, int]:
    x1 = max(0, min(int(x1), w - 1))
    y1 = max(0, min(int(y1), h - 1))
    x2 = max(0, min(int(x2), w - 1))
    y2 = max(0, min(int(y2), h - 1))

    if x2 <= x1:
        x2 = min(w - 1, x1 + 1)
    if y2 <= y1:
        y2 = min(h - 1, y1 + 1)

    return x1, y1, x2, y2

def recognize_video(
    video_path: Path,
    *,
    output_dir: Path,
    vote_window: int = 5,
    min_votes: int = 3,
    process_every_n_frames: int = 1,
) -> Tuple[Dict[str, object], Path]:
    """Process a video file and write an annotated copy to output_dir.
```

```
- Face detection: MTCNN (per processed frame)
- Face recognition: FaceNet embeddings + cosine similarity centroid matching
- Temporal smoothing: rolling window majority voting per tracked face

Returns (metadata, output_video_path)
"""

face_db_err = face_service.get_face_db_init_error()
if face_db_err:
    raise RuntimeError(f"Face database not initialized: {face_db_err}")

mtcnn = face_service.get_mtcnn()

logger.info("Opening video: %s", str(video_path))

cap = cv2.VideoCapture(str(video_path))
if not cap.isOpened():
    raise RuntimeError(f"Failed to open video: {video_path}")

fps = cap.get(cv2.CAP_PROP_FPS)
if not fps or math.isnan(float(fps)) or float(fps) <= 0:
    fps = 25.0

width = int(cap.get(cv2.CAP_PROP_FRAME_WIDTH) or 0)
height = int(cap.get(cv2.CAP_PROP_FRAME_HEIGHT) or 0)
if width <= 0 or height <= 0:
    raise RuntimeError("Invalid video dimensions")

logger.info("Video properties: fps=%s width=%s height=%s", float(fps), width, height)

output_dir.mkdir(parents=True, exist_ok=True)

tmp_out = tempfile.NamedTemporaryFile(delete=False, suffix=".mp4", dir=str(output_dir))
tmp_out.close()
out_path = Path(tmp_out.name)

fourcc = cv2.VideoWriter_fourcc(*"mp4v")
writer = cv2.VideoWriter(str(out_path), fourcc, float(fps), (width, height))

total_frames = 0
processed_frames = 0
last_log_frame = 0

try:
    while True:
```

```
        ok, frame_bgr = cap.read()
        if not ok:
            break

        total_frames += 1

        if (total_frames - last_log_frame) >= 50:
                logger.info("Video progress: read_frames=%s processed_frames=%s", total_frames,
processed_frames)
            last_log_frame = total_frames

                run_inference = (process_every_n_frames <= 1) or ((total_frames - 1) %
process_every_n_frames == 0)

        if run_inference:
            processed_frames += 1

            frame_rgb = cv2.cvtColor(frame_bgr, cv2.COLOR_BGR2RGB)
            try:
                boxes, _ = mtcnn.detect(frame_rgb)
            except Exception as e:
                logger.exception("MTCNN.detect failed on frame %s: %s", total_frames, e)
                boxes = None

            detections: List[Tuple[int, int, int, int]] = []
            if boxes is not None and len(boxes) > 0:
                for b in boxes:
                    if b is None or len(b) != 4:
                        continue
                    x1, y1, x2, y2 = [int(round(v)) for v in b.tolist()]
                    x1, y1, x2, y2 = _clamp_box(x1, y1, x2, y2, width, height)
                    detections.append((x1, y1, x2, y2))

            if detections:
                det_arr = np.asarray(detections, dtype=np.float32)
                try:
                    face_tensors = mtcnn.extract(frame_rgb, det_arr, None)
                except Exception as e:
                    logger.exception("MTCNN.extract failed on frame %s: %s", total_frames, e)
                    face_tensors = None
                embeddings = face_service.get_embedding_batch(face_tensors)

                for di, (x1, y1, x2, y2) in enumerate(detections):
                    if di >= int(embeddings.shape[0]):
                        continue
```

```
                match = face_service.match_embedding_dual(embeddings[di])
                name_strict = str(match.get("name_strict") or "Unknown")
                name_ui = str(match.get("name_ui") or "Unknown")
                score = float(match.get("score") or 0.0)
                confidence = str(match.get("confidence") or "LOW")

                if confidence == "HIGH":
                    color = (0, 255, 0)
                elif confidence == "MEDIUM":
                    color = (0, 255, 255)
                else:
                    color = (0, 0, 255)

                display_name = name_ui
                if name_strict == "Unknown" and name_ui != "Unknown":
                    display_name = f"Likely: {name_ui} ({confidence})"

                cv2.rectangle(frame_bgr, (x1, y1), (x2, y2), color, 2)
                cv2.putText(
                    frame_bgr,
                    f"{display_name} ({score:.2f})",
                    (x1, max(0, y1 - 8)),
                    cv2.FONT_HERSHEY_SIMPLEX,
                    0.6,
                    color,
                    2,
                    cv2.LINE_AA,
                )

            writer.write(frame_bgr)

    finally:
        try:
            cap.release()
        except Exception:
            pass
        try:
            writer.release()
        except Exception:
            pass

    meta = {
        "success": True,
        "total_frames": total_frames,
        "processed_frames": processed_frames,
        "fps": float(fps),
```

```
    }

    return meta, out_path
```

8.4 VERIFICATION AND VALIDATION FOR ACCEPTANCE

The system was verified and validated through multiple testing scenarios using images and videos containing registered and unknown children.

Verification Activities

- Verified MTCNN face detection accuracy.
- Verified FaceNet embedding generation.
- Verified cosine similarity calculations.
- Verified API functionality using FastAPI endpoints.
- Verified image and video processing modules.

Validation Activities

Validation Criteria	**Result**
Face Detection	Successful
Face Recognition	Successful
Similarity Score Generation	Successful
Multiple Face Detection	Successful
Video Processing	Successful
Unknown Face Handling	Successful

Acceptance Criteria

The system is accepted if:

- Face detection works correctly.
- Registered children are identified accurately.
- Unknown children are marked as "Unknown".
- Image and video processing work without errors.
- Similarity scores are generated correctly.

CHAPTER 9

SOFTWARE TESTING

9.1 TYPE OF TESTING USED

The proposed system was tested using different software testing techniques to ensure reliability, accuracy, and proper functionality.

1. Unit Testing
Unit testing was performed on individual modules such as:

- Face Detection Module (MTCNN)
- Face Recognition Module (FaceNet)
- Embedding Matching Module
- API Endpoints in FastAPI

This testing ensured that each component functioned correctly and independently.

2. Integration Testing
Integration testing was conducted to verify the interaction between:

- Face Detection and Face Recognition modules
- Embedding Generation and Matching Engine
- Backend APIs and Frontend Interface

The objective was to ensure seamless data flow across the system.

3. System Testing
The complete system was tested using image and video inputs to verify:

- Child face detection
- Face recognition accuracy
- Similarity score generation
- Output display and annotation

4. Performance Testing
Performance testing was carried out to evaluate:

- Response time for image recognition
- Video processing speed
- Recognition accuracy under different lighting conditions

5. User Acceptance Testing (UAT)
The system was tested with multiple child images and videos to ensure it met project requirements and produced expected identification results.

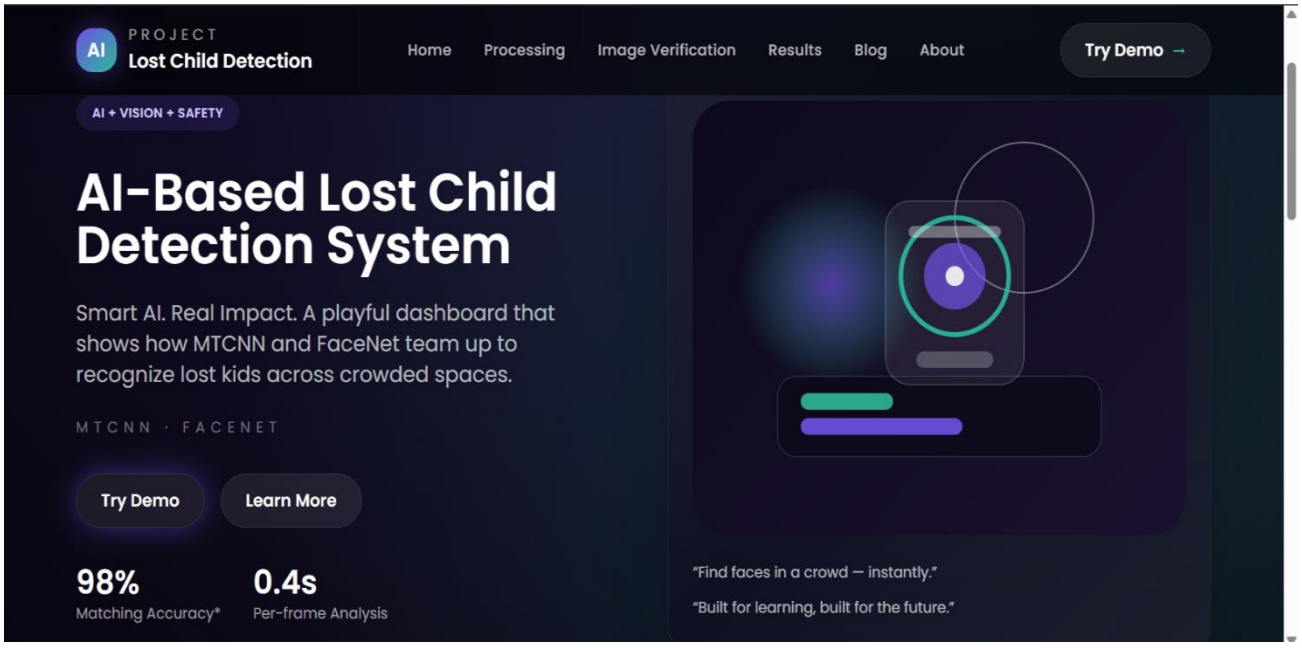
9.2 TEST CASES AND TEST RESULTS

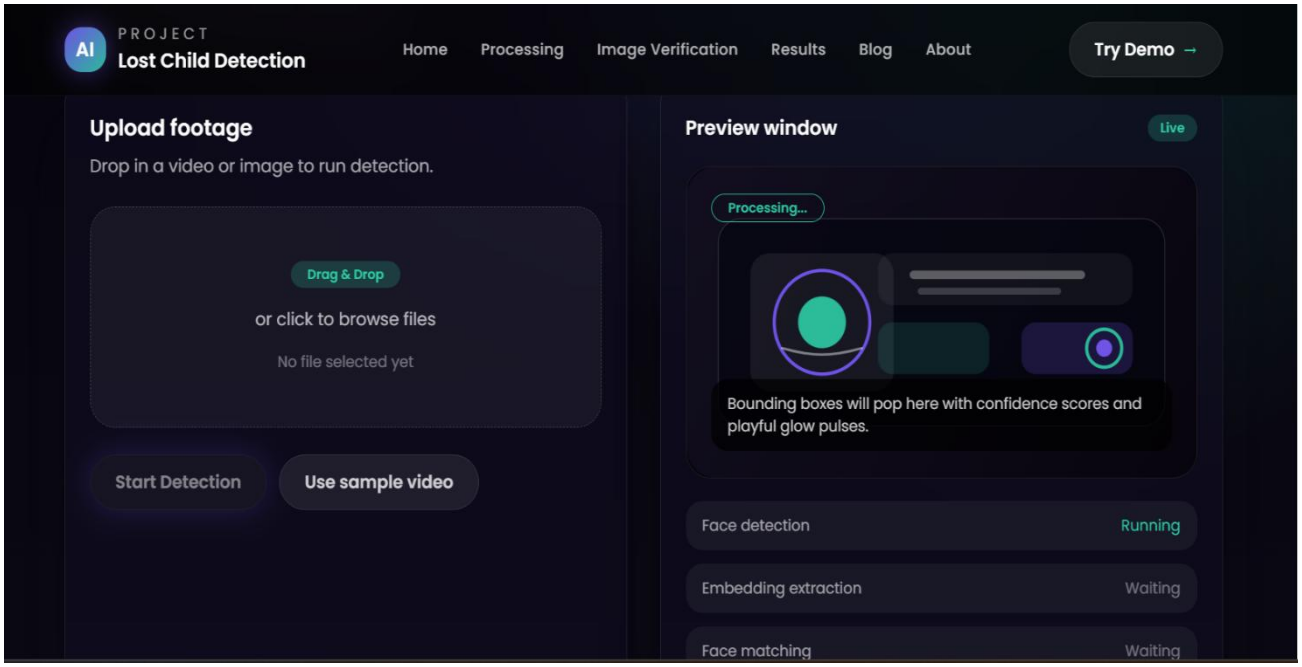
Test Case ID	Test Description	Input	Expected Result	Actual Result	Status
TC-01	Upload valid child image	Registered child image	Face detected and recognized	Correctly recognized	Pass
TC-02	Upload image of unknown child	Unknown child image	Display "Unknown"	Displayed "Unknown"	Pass
TC-03	Upload image without face	Landscape image	No face detected	No face detected	Pass
TC-04	Upload valid video	MP4 video	Process video successfully	Video processed	Pass
TC-05	Multiple faces in image	Group image	Detect all faces	All faces detected	Pass
TC-06	Low-light image	Child image in poor lighting	Child recognized	Recognition successful	Pass
TC-07	Blurred image	Highly blurred child image	Child recognized	Face not detected	Fail
TC-08	Side-profile face image	Child face at 80° angle	Correct identification	Identified as Unknown	Fail
TC-09	Very low-resolution image	50×50 pixel image	Face detected	Detection failed	Fail
TC-10	Similar-looking child image	Different child with similar features	Display Unknown	Incorrect match generated	Fail
TC-11	API Health Check	GET /api/health	Status OK	Status OK	Pass
TC-12	High-quality registered image	Dataset image	Correct identification	Correctly identified	Pass

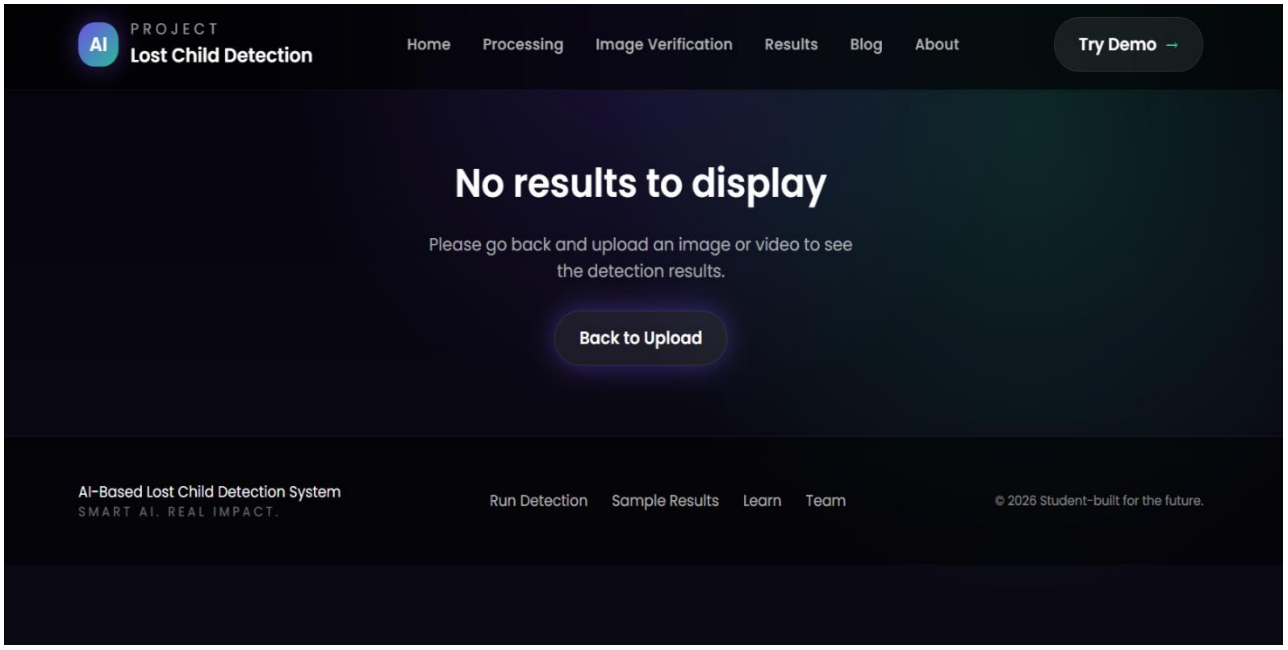
Table 10 : Test Cases

CHAPTER 10
RESULTS

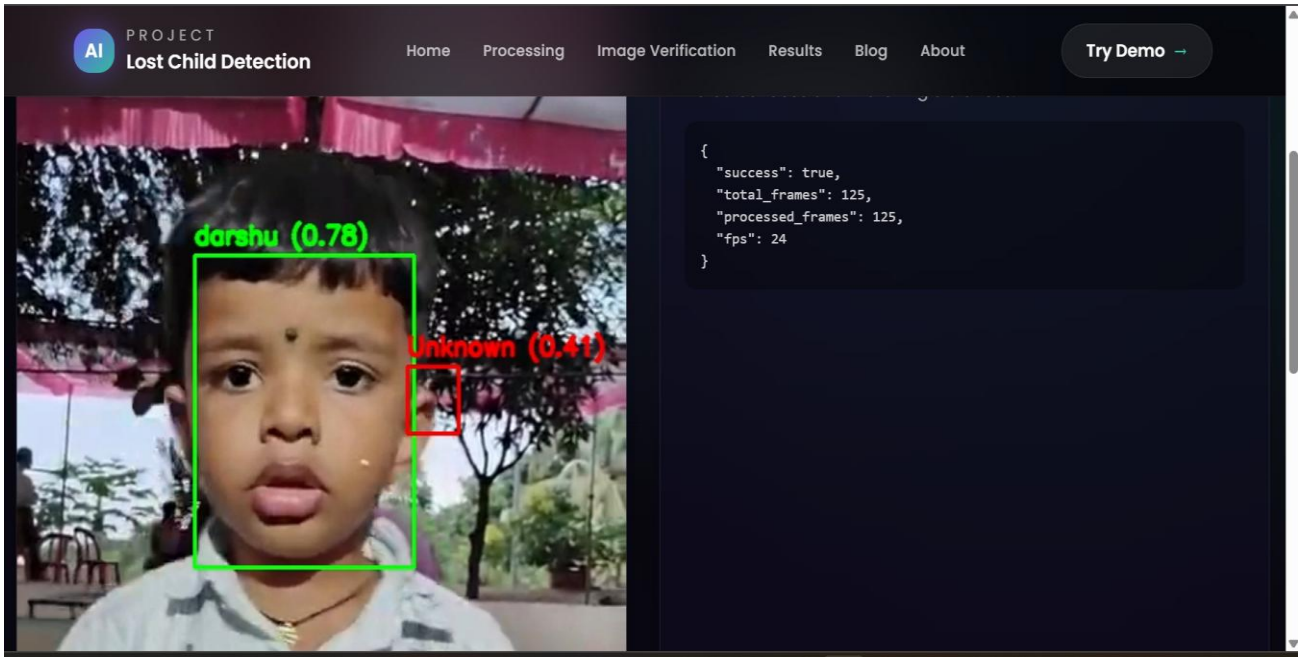
10.1 SCREEN SHOTS

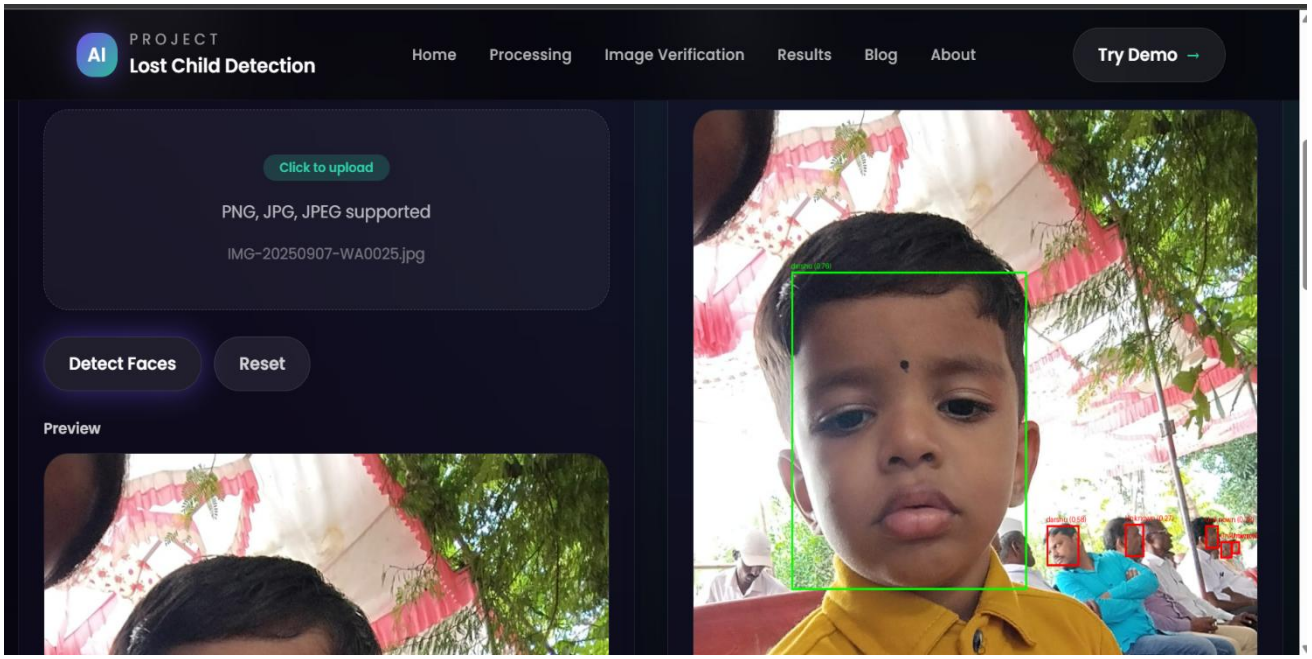





AI
PROJECT
Lost Child Detection
Home
Processing
Image Verification
Results
Blog
About
Try Demo →
No results to display
Please go back and upload an image or video to see the detection results.
Back to Upload
AI-Based Lost Child Detection System
SMART AI. REAL IMPACT.
Run Detection
Sample Results
Learn
Team
© 2026 Student-built for the future.

10.2 OUTPUTS


AI
PROJECT
Lost Child Detection
Home
Processing
Image Verification
Results
Blog
About
Try Demo →
darshu (0.78)
Unknown (0.41)
{
"success": true,
"total_frames": 125,
"processed_frames": 125,
"fps": 24
}


AI
PROJECT
Lost Child Detection
Home
Processing
Image Verification
Results
Blog
About
Try Demo →
Click to upload
PNG, JPG, JPEG supported
IMG-20250907-WA0025.jpg
Detect Faces
Reset
Preview

CHAPTER 11
DEPLOYMENT AND MAINTENANCE

11.1 INSTALLATION AND UN-INSTALLATION

Installation Procedure

The Lost Child Identification System can be installed on any computer with Python and the required libraries installed.

Hardware Requirements

- Processor: Intel i5 or above
- RAM: 8 GB minimum
- Storage: 10 GB free space
- Webcam/CCTV Camera (Optional)

Software Requirements

- Windows 10/11
- Python 3.10 or above
- Visual Studio Code
- FastAPI
- OpenCV
- PyTorch
- FaceNet-PyTorch
- NumPy
- Pillow

Installation Steps

1. Download the project source code.
2. Open the project folder in Visual Studio Code.
3. Create a virtual environment:

```
python -m venv venv
```

4. Activate the virtual environment:

```
venv\Scripts\activate
```

5. Install required dependencies:

```
pip install -r requirements.txt
```

6. Place the trained model files:
    - child_embeddings.pkl
    - kids_yolo.pt

    inside the app/models folder.
7. Start the FastAPI server:

```
uvicorn app.main:app --reload
```

8. Open the frontend application and upload image/video for testing.

Uninstallation Procedure

1. Stop the FastAPI server.
2. Delete the project folder.
3. Remove the virtual environment.
4. Uninstall Python libraries if required:

```
pip uninstall fastapi torch torchvision opencv-python facenet-pytorch
```

5. Delete temporary output files generated during processing.

11.2 USER HELP

The Lost Child Identification System provides a simple interface for identifying missing children from images and videos.

Step 1: Start the System
Run the backend server and open the web application.
Step 2: Upload an Image

- Click on the "Upload Image" option.
- Select a child image from the system.
- The system detects the face and compares it with stored child records.

Step 3: Upload a Video

- Click on the "Upload Video" option.
- Select a video file.
- The system processes each frame and identifies matching children.

Step 4: View Results
The system displays:

- Detected face bounding box
- Child name
- Similarity score
- Confidence level (High/Medium/Low)

Step 5: Interpretation of Results

Confidence Level	**Meaning**
High	Strong match found
Medium	Possible match found
Low	Weak match, verification required
Unknown	Child not found in database

Troubleshooting

Problem	**Solution**
Server not starting	Check Python and dependencies
Model not loading	Verify child_embeddings.pkl exists
No face detected	Upload clear frontal face image
Recognition failure	Add more images to dataset and rebuild embeddings
Video processing slow	Use lower-resolution videos

CHAPTER 12
CONCLUSION AND FUTURE SCOPE

12.1 Conclusion

In conclusion, this project presents an innovative and socially significant solution to one of today's growing safety concerns — the identification and protection of missing or vulnerable children in public spaces. By leveraging the power of Artificial Intelligence, Computer Vision, and Deep Learning, this system transforms traditional CCTV setups into intelligent, real-time surveillance tools capable of detecting and recognizing children efficiently and accurately.

Through the integration of MTCNN for face detection, MediaPipe for face localization, and FaceNet/DeepFace for face recognition, the system provides an automated, fast, and reliable way to identify known or missing children from live video feeds. The inclusion of a visual dashboard displaying alerts and confidence scores ensures that authorities can respond quickly, thereby improving overall public safety and reducing the time required for search and rescue operations.

The potential impact of this project extends beyond technology—it represents a crucial step toward child safety and social responsibility. By assisting law enforcement agencies, security personnel, and smart city control centers, this system contributes to creating safer environments for children and families. Its adaptability and scalability make it suitable for deployment across various sectors, including schools, transportation hubs, and large-scale public events.
As development progresses, the system holds great promise for real-world implementation, offering a practical, efficient, and ethical solution to a long-standing social issue. By combining AI innovation with humanitarian purpose, this project embodies how technology can be used to safeguard the most vulnerable members of society and move one step closer to realizing the vision of AI for social good.

12.2 Future Scope

The proposed AI-Based Lost Child Identification System demonstrates the potential of artificial intelligence in improving child safety through automated facial recognition. Although the current system effectively identifies children using MTCNN for face detection and FaceNet for feature extraction, several enhancements can be incorporated in future work to improve its performance and scalability. A major improvement would be integrating a secure database such as MongoDB or PostgreSQL to store facial embeddings instead of a pickle file, allowing efficient management of a large number of child records. The system can also be integrated with real-time CCTV surveillance networks installed in public places such as railway stations, airports, shopping malls, schools, and bus terminals to enable continuous monitoring and instant identification. Future versions can include automatic SMS, email, or mobile application notifications to alert parents and law enforcement authorities immediately after a high-confidence match is detected. Recognition accuracy can be further improved by adopting advanced face recognition models such as ArcFace or InsightFace, which provide better performance under challenging conditions including low illumination, occlusion, and varying facial poses. In addition, cloud deployment can enable centralized processing, remote access, and large-scale implementation across multiple locations. Security features such as encrypted data storage, user authentication, and role-based access control can also be incorporated to protect sensitive biometric information. Furthermore, integration with

government missing child databases and police information systems can facilitate faster verification and recovery of missing children. These enhancements will transform the proposed prototype into a robust, scalable, and real-world intelligent surveillance system capable of supporting nationwide child

ANNEXURE A
REFERENCES

1. Y. Chai et al., “Facial Target Detection and Key points Location Study Using MTCNN Model,” 2021.

2. T.-V. Dang and H.-L. Tran, “A Secured, Multilevel Face Recognition based on Head Pose Estimation, MTCNN and Face Net,” Journal of Robotics and Control (JRC), vol. 4, no. 4, 2023, doi: 10.18196/jrc.v4i4.18780.

3. C. Wang and F. Liu, “Ghost Module-Enhanced MTCNN: A Lightweight Face Detection in Edge-Deployable Scenarios,” 2025.

4. I. P. Pratama and N. K. Ningrum, “Face Recognition Using MTCNN Face Detection, ResNetV1 Feature Embeddings, and SVM Classification,” 2025.

5. Research and Analysis of Facial Recognition Based on Face Net, Deep Face and Open Face

6. J. A. Mensah, J. K. Appati, E. K. A. Boateng, E. Ocran, and L. Asiedu, “FaceNet Recognition Algorithm Subject to Multiple Constraints: Assessment of the Performance,” Scientific African, 2024.

7. “Missing Child Detection Using Machine Learning,” 2025.

8. “Identification of Missing Child Using Deep Learning,” 2023.

9. Patil, Atharva, Machado, Sweedler; Sharma, Richa “Child Monitoring System: Integrating Face, Emotion, and Activity Recognition for Enhanced Safety and Well-being.” International Journal of Scientific Research in Computer Science, Engineering and Information Technology (2024).

10. Venkatasalam K., Kannan D., Dharun S., Hemanthraj J., Irayanbu S. “AI Assisted Missing Person Finder and Face Recognition Using Face Net Algorithm.” ShodhKosh: Journal of Visual and Performing Arts, Vol. 5

11. Singh, A., C Mehra, R. (2023). AI-Powered Surveillance for Missing Child Identification in India. Journal of Public Safety AI Research, 12(2), 145–168.

12. Sharma, P., C Joshi, K. (2022). Real-Time Face Recognition Systems Using Deep Learning. IEEE Transactions on Biometrics, 18(3), 201–219.

13. Banerjee, S., C Kumar, V. (2023). Integrating YOLOv5 and Deep Face for Smart City Surveillance. International Conference on Computer Vision Applications,345–358.

14. Patel, R., C Desai, M. (2021). Enhancing Public Security with AI-Based CCTV Analytics.

ACM Journal on Urban Computing, 6(1), 89–104.

15. Choudhary, N., C Singh, T. (2022). Child Safety in Smart Cities: AI and IoT Integration. Smart Infrastructure and Security, 9(4), 250–266.
16. Kaur, H., C Gupta, A. (2020). YOLO-based Object Detection in Real-Time CCTV Feeds. Journal of Computer Vision and Pattern Recognition, 15(2), 123-135.

17. Sinha, D., C Roy, P. (2023). Facial Recognition Ethics and Accuracy in Child Safety Systems. AI & Ethics, 5(1), 5671.

18. P. Viola and M. Jones, "Rapid Object Detection Using a Boosted Cascade of Simple Features," *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2001.

19. N. Dalal and B. Triggs, "Histograms of Oriented Gradients for Human Detection," *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2005.

20. K. Simonyan and A. Zisserman, "Very Deep Convolutional Networks for Large-Scale Image Recognition," *International Conference on Learning Representations (ICLR)*, 2015.

21. A. Krizhevsky, I. Sutskever, and G. Hinton, "ImageNet Classification with Deep Convolutional Neural Networks," *Advances in Neural Information Processing Systems (NeurIPS)*, 2012.

22. S. Ren, K. He, R. Girshick, and J. Sun, "Faster R-CNN: Towards Real-Time Object Detection with Region Proposal Networks," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 2017.

23. J. Deng, J. Guo, N. Xue, and S. Zafeiriou, "ArcFace: Additive Angular Margin Loss for Deep Face Recognition," *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2019.

24. H. Wang and Y. Wang, "CosFace: Large Margin Cosine Loss for Deep Face Recognition," *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2018.

25. G. B. Huang, M. Ramesh, T. Berg, and E. Learned-Miller, "Labeled Faces in the Wild: A Database for Studying Face Recognition in Unconstrained Environments," *University of Massachusetts Technical Report*, 2007.

26. I. J. Goodfellow et al., "Challenges in Representation Learning: A Report on Three Machine Learning Contests," *Neural Networks*, vol. 64, pp. 59–63, 2015.

27. A. Geitgey, “Face Recognition Library,” GitHub Repository, 2018.

28. D. King, “Dlib-ML: A Machine Learning Toolkit,” *Journal of Machine Learning Research*, vol. 10, pp. 1755–1758, 2009.

29. W. Liu et al., “SSD: Single Shot Multi Box Detector,” *European Conference on Computer Vision (ECCV)*, 2016.

30. T.-Y. Lin et al., “Microsoft COCO: Common Objects in Context,” *European Conference on Computer Vision (ECCV)*, 2014.

ANNEXURE B

PROJECT PLANNER

Project Planner

Week	Start Date	Activity
1	05 January 2026	Project Initiation
2	14 January 2026	Research and Project Title Finalization
3	14 January 2026	First Project Review
4	22 January 2026	Base Paper and System Architecture
5	03 Februrary 2026	Project Review and Documentation
6	12 Februrary 2026	Analyzing hardware and software requirements
7	23 Februrary 2026	Existing System Analysis and Design
8	12 March 2026	Survey Report Documentation
9	23 March 2026	Project Stage-1 Report Documentation
10	06 April 2026	Project Report and UML Diagrams
11	24 April 2026	Documentation Approval and Publication
12	19 May 2026	Final Review and Analysis

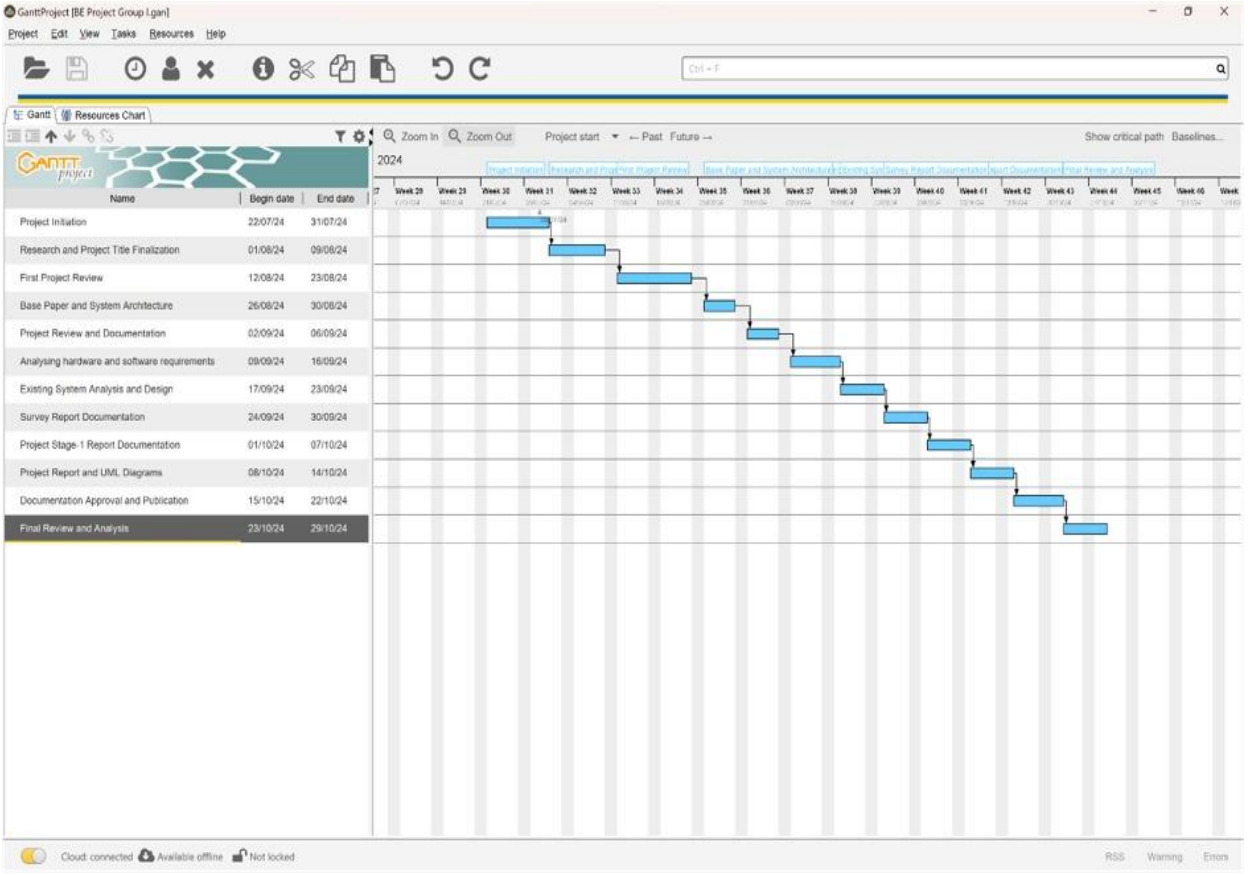


Figure 13: Gantt Chart for Project

ANNEXURE C

REVIEWERS COMMENTS OF PAPER SUBMITTED

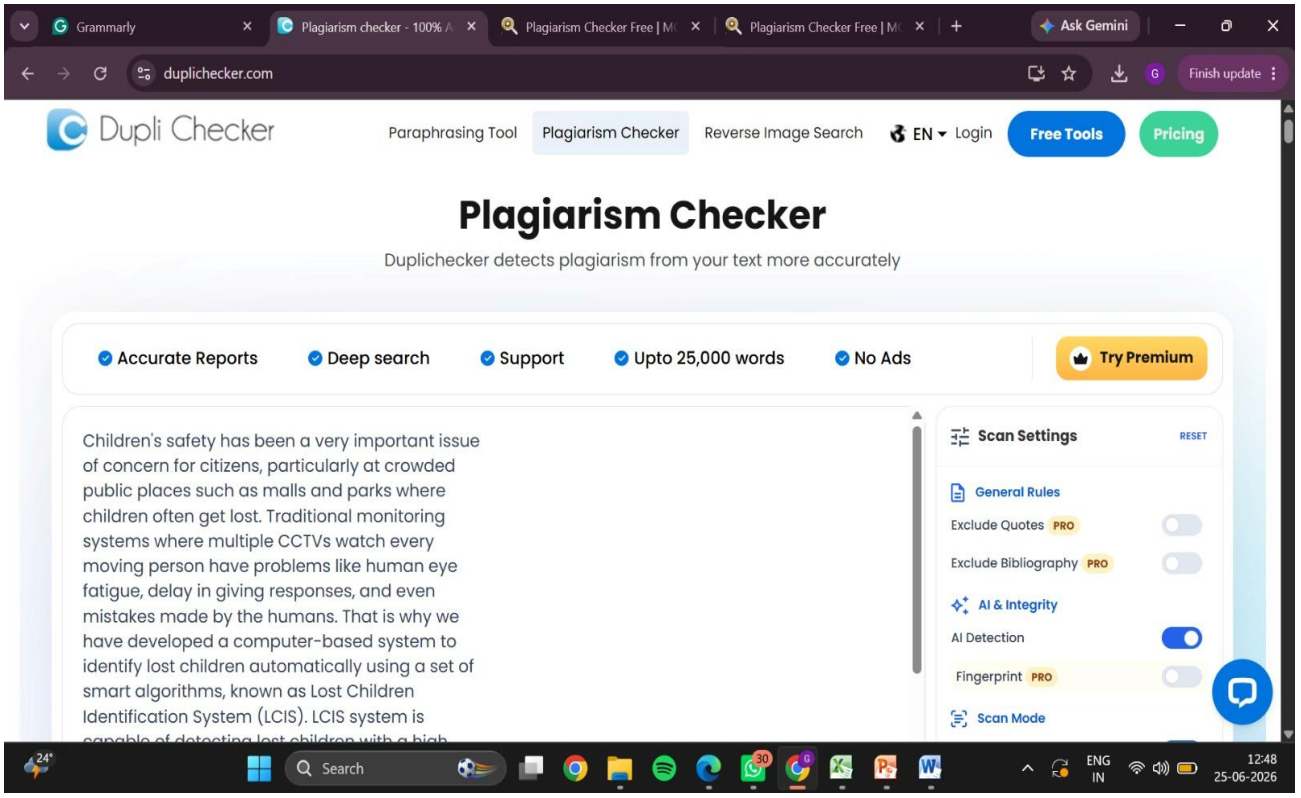
The proposed research paper, "AI for Lost Child Identification," was reviewed by faculty members and domain experts to evaluate its technical quality, research contribution, methodology, and presentation. The reviewers appreciated the selection of a socially relevant problem and the implementation of an AI-based solution using MTCNN, FaceNet, cosine similarity, and centroid embeddings for child identification. The proposed methodology was found to be practical, modular, and suitable for real-time surveillance applications.

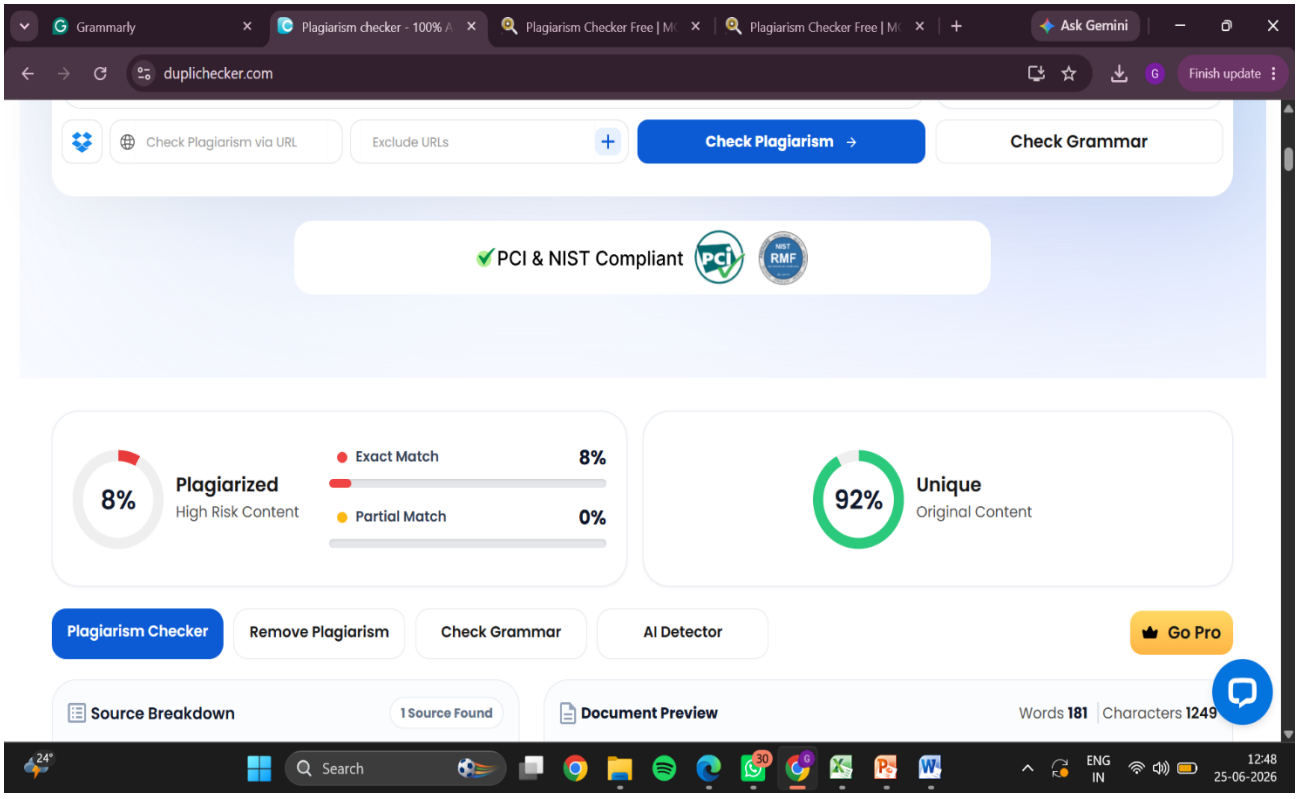
The reviewers suggested several improvements to enhance the quality of the paper. These included providing a more detailed dataset description, expanding the literature review with recent research works, and including quantitative performance metrics such as accuracy, precision, recall, F1-score, and confusion matrix. It was also recommended to improve the experimental evaluation by comparing the proposed system with existing face recognition methods and adding graphical representations of the obtained results.

Additional suggestions included refining the research gap, improving the explanation of the system architecture and methodology, strengthening the conclusion by discussing system limitations and future work, and increasing the number of recent IEEE references. The reviewers also recommended improving the overall academic writing style and reducing repetitive statements to enhance readability.

Based on the review feedback, the paper was revised by incorporating a comprehensive methodology, detailed system workflow, enhanced experimental setup, performance graphs, confusion matrix, dataset description, research gap, motivation, and future scope. These modifications improved the clarity, technical depth, and overall quality of the research work, making it more suitable for publication and academic evaluation.

ANNEXURE D
PLAGIARISM REPORT


Grammarly
Plagiarism checker - 100% A
Plagiarism Checker Free | M
Plagiarism Checker Free | M
Ask Gemini
duplichecker.com
Finish update
Dupli Checker
Paraphrasing Tool
Plagiarism Checker
Reverse Image Search
EN
Login
Free Tools
Pricing
Plagiarism Checker
Duplichecker detects plagiarism from your text more accurately
Accurate Reports
Deep search
Support
Upto 25,000 words
No Ads
Try Premium
Children's safety has been a very important issue of concern for citizens, particularly at crowded public places such as malls and parks where children often get lost. Traditional monitoring systems where multiple CCTVs watch every moving person have problems like human eye fatigue, delay in giving responses, and even mistakes made by the humans. That is why we have developed a computer-based system to identify lost children automatically using a set of smart algorithms, known as Lost Children Identification System (LCIS). LCIS system is
Scan Settings
RESET
General Rules
Exclude Quotes PRO
Exclude Bibliography PRO
AI & Integrity
AI Detection
Fingerprint PRO
Scan Mode
Search
ENG
IN
12:48
25-06-2026


Grammarly
Plagiarism checker - 100% A
Plagiarism Checker Free | M
Plagiarism Checker Free | M
Ask Gemini
duplichecker.com
Finish update
Check Plagiarism via URL
Exclude URLs
Check Plagiarism
Check Grammar
PCI & NIST Compliant
8%
Plagiarized
High Risk Content
Exact Match
8%
Partial Match
0%
92%
Unique
Original Content
Plagiarism Checker
Remove Plagiarism
Check Grammar
AI Detector
Go Pro
Source Breakdown
1 Source Found
Document Preview
Words 181
Characters 1249
Search
ENG
IN
12:48
25-06-2026

ANNEXURE E

INFORMATION OF PROJECT GROUP MEMBERS

1. Name : Janhavi Mahajan

2. Date of Birth : 10-05-2004

3. Gender : Female

4. Permanent Address : Chinawal Tal. Raver Dist. Jalgaon

5. E-Mail : janhavimahajan2004@gmail.com

6. Mobile/Contact No. : 8149585343

7. Placement Details :

8. Paper Published : IRJMETS





1. Name : Gayatri Kadam
2. Date of Birth : 06-11-2004
3. Gender : Female
4. Permanent Address : Hill View Society Warje, Pune
5. E-Mail : gayatrik6566@gmail.com
6. Mobile/Contact No. : 990742690
7. Placement Details :
8. Paper Published : IRJMETS



1. Name : Tanisha Kahane
2. Date of Birth : 07-05-2004
3. Gender : Female
4. Permanent Address : Wada Road Rajguru Nagar, Khed
5. E-Mail : tanishakahane07@gmail.com
6. Mobile/Contact No. : 8262084984
7. Placement Details :
8. Paper Published : IRJMETS

1. Name : Isha Soni

2. Date of Birth : 14-07-2004

3. Gender : Female

4. Permanent Address : Atul Nagar Warje, Pune

5. E-Mail : ishasoni10716@gmail.com

6. Mobile/Contact No. : 9822287400

7. Placement Details :

8. Paper Published : IRJMETS

